

Comprehensive Guide to C Programming

Table of Contents

1. [Introduction to C Programming](#)
2. [C Programming in the Era of AI](#)
3. [Programming Environment and GCC Installation](#)
4. [Program Structure, Characteristics and Errors](#)
5. [Output Functions](#)
6. [Variables and Data Types](#)
7. [Input Functions, Range of Values, and Qualifiers](#)
8. [Operators and Expressions](#)
9. [Programming using Variables, Data Types, Operators, and I/O Functions](#)
10. [Control Structures - Selection Statements and Looping](#)
11. [Problem Solving using Control Structures](#)
12. [Single Character Input and Output](#)
13. [Problem Solving using Control Structures and Character I/O](#)
14. [Language Specifications and Behaviors](#)
15. [Coding Standards and Guidelines to Avoid Errors](#)

Introduction to C Programming

C is a general-purpose, procedural computer programming language developed in the early 1970s by Dennis Ritchie at Bell Labs. It was designed to be a language that was both efficient and flexible, allowing close control over hardware resources while maintaining portability.

Key Features of C

- **Mid-level language:** Combines elements of both high-level and low-level languages
- **Portability:** Code written in C can be compiled for different platforms with minimal changes
- **Efficiency:** Provides access to memory manipulation and hardware control
- **Modularity:** Supports structured programming through functions
- **Rich library support:** Comes with standard libraries for common operations
- **Foundation for other languages:** Many modern languages like C++, Java, and Python have drawn syntax and concepts from C

Why C Remains Relevant

Despite being several decades old, C continues to be widely used for:

- Operating systems development
- Embedded systems programming
- System utilities
- Performance-critical applications
- Legacy system maintenance
- Foundation for learning programming concepts

C Programming in the Era of AI

While Python and other high-level languages dominate AI development, C remains crucial in the AI ecosystem:

C's Role in Modern AI

- **Performance-critical AI components:** When speed matters, C is often used to optimize computationally intensive operations
- **Embedded AI applications:** For IoT devices and other resource-constrained environments
- **AI frameworks:** Many AI libraries have C backends (e.g., TensorFlow's core operations)
- **System-level AI integration:** Integrating AI capabilities into operating systems and low-level software

Examples of C in AI Contexts

- Optimized matrix operations in deep learning libraries
- Real-time image processing for computer vision
- Resource-efficient inference engines for edge devices
- Kernel-level integration of AI capabilities

Programming Environment and GCC Installation

Setting Up a C Development Environment

Windows

1. Installing MinGW (Minimalist GNU for Windows):

- Download the MinGW installer from <https://osdn.net/projects/mingw/>
- Run the installer and select the C compiler package
- Add MinGW's bin directory to your PATH environment variable

macOS

1. Installing Xcode Command Line Tools:

```
xcode-select --install
```

This installs the necessary tools including GCC.

Linux

1. Installing GCC and development tools:

```
# For Ubuntu/Debian:
sudo apt-get update
sudo apt-get install build-essential

# For Fedora:
sudo dnf install gcc

# For CentOS/RHEL:
sudo yum group install "Development Tools"
```

Verifying Installation

To verify that GCC is properly installed, open a terminal or command prompt and run:

```
gcc --version
```

Integrated Development Environments (IDEs)

While a simple text editor and terminal can be sufficient, IDEs enhance productivity:

1. Visual Studio Code:

- Free, lightweight, and extensible
- Install the C/C++ extension for enhanced features

2. Code::Blocks:

- Open-source, cross-platform IDE specifically designed for C/C++
- Comes with built-in compiler support

3. Dev-C++:

- Simple IDE popular for beginners on Windows

4. Eclipse with CDT:

- Full-featured IDE for C/C++ development

Program Development Life Cycle (PDLC)

The typical program development cycle in C follows these steps:

1. **Requirements Analysis:** Understanding what the program needs to accomplish
2. **Design:** Planning the program structure, algorithms, and data structures
3. **Implementation:** Writing the code in C
4. **Compilation:** Converting source code to machine code

```
gcc program.c -o program
```

5. **Testing:** Running the program to check if it meets requirements
6. **Debugging:** Finding and fixing errors
7. **Deployment:** Distributing the compiled program to users
8. **Maintenance:** Updating the program to fix bugs or add features

Program Structure, Characteristics and Errors

Basic Structure of a C Program

Every C program follows a basic structure:

```
/* Preprocessor directives */
#include <stdio.h>

/* Function declarations or prototypes (if any) */
double calculateArea(double radius);

/* Global variables (if any) */
const double PI = 3.14159;

/* Main function - program execution starts here */
int main() {
    /* Local variable declarations */
    double radius = 5.0;
    double area;

    /* Statements and expressions */
    area = calculateArea(radius);
    printf("Area of circle with radius %.2f is %.2f\n", radius, area);

    /* Return statement */
    return 0;
}

/* Function definitions */
double calculateArea(double radius) {
    return PI * radius * radius;
}
```

Key Components of a C Program

1. **Preprocessor Directives:** Instructions to the compiler before actual compilation begins
 - Include files: `#include <header_file.h>`
 - Macro definitions: `#define PI 3.14159`
 - Conditional compilation: `#ifdef` , `#ifndef` , `#endif`
2. **Functions:** Building blocks of a C program
 - Every program must have a `main()` function where execution begins
 - Each function has a return type, name, parameters, and body
3. **Variables:** Named storage for data
 - Must be declared before use
 - Can be local (inside functions) or global (outside functions)
4. **Statements:** Instructions that perform actions
 - Assignment statements
 - Function calls
 - Control statements
 - Each statement typically ends with a semicolon

Common Errors in C Programming

1. **Syntax Errors:** Detected during compilation

- Missing semicolons
- Undeclared variables
- Mismatched parentheses or braces
- Example:

```
int main() {  
    printf("Hello, world!\n") // Missing semicolon  
    return 0;  
}
```

2. **Semantic Errors:** Logical errors that don't violate syntax

- Using uninitialized variables
- Type mismatches in assignments
- Off-by-one errors in loops
- Example:

```
int main() {  
    int sum = 0;  
    // Should be i <= 10 to include 10  
    for(int i = 1; i < 10; i++) {  
        sum += i;  
    }  
}
```

```

        printf("Sum from 1 to 10: %d\n", sum);
        return 0;
    }

```

3. Runtime Errors: Occur during program execution

- Division by zero
- Memory access violations
- Stack overflow
- Example:

```

int main() {
    int a = 10;
    int b = 0;
    int result = a / b; // Division by zero - runtime error
    printf("Result: %d\n", result);
    return 0;
}

```

4. Linker Errors: Occur when the linker can't resolve references

- Undefined references to functions
- Multiple definitions of functions or variables
- Example:

```

// In main.c
int main() {
    int result = calculate(10); // Function is declared but not defined
    printf("Result: %d\n", result);
    return 0;
}

// No implementation of calculate() function

```

Compilation Process in C

The process of converting C source code to an executable involves four main steps:

1. **Preprocessing:** Handling all preprocessor directives
 - Includes header files
 - Expands macros
 - Removes comments
2. **Compilation:** Converting preprocessed code to assembly code
 - Checks syntax
 - Generates assembly instructions
3. **Assembly:** Converting assembly code to object code
 - Generates machine code in object files

4. Linking: Combining object files into an executable

- Resolves references between files
- Includes code from libraries

Output Functions

C provides several ways to display output to the user, primarily through the functions defined in the `stdio.h` header file.

The `printf()` Function

The most commonly used output function is `printf()`, which allows formatted output:

```
#include <stdio.h>

int main() {
    printf("Hello, world!\n"); // Simple string output

    int age = 25;
    printf("I am %d years old.\n", age); // Integer output

    float height = 5.9;
    printf("My height is %.1f feet.\n", height); // Float output with precision

    char grade = 'A';
    printf("I got %c grade in programming.\n", grade); // Character output

    char name[] = "John";
    printf("My name is %s.\n", name); // String output

    printf("Multiple values: %d, %.2f, %c\n", age, height, grade); // Multiple values

    return 0;
}
```

Format Specifiers

Format specifiers allow you to format different data types in the output:

Specifier	Data Type	Example
%d , %i	Integer	<code>printf("%d", 42);</code>
%f	Float/Double	<code>printf("%f", 3.14);</code>
%e , %E	Scientific notation	<code>printf("%e", 3000000.0);</code> → 3.000000e+06
%c	Character	<code>printf("%c", 'A');</code>

Specifier	Data Type	Example
%s	String	printf("%s", "Hello");
%x , %X	Hexadecimal	printf("%x", 255); → ff
%o	Octal	printf("%o", 64); → 100
%p	Pointer address	printf("%p", &variable);
%u	Unsigned integer	printf("%u", 42);
%%	Percent sign	printf("50%%"); → 50%

Format Specifier Modifiers

You can modify format specifiers to control the output format:

1. Field Width:

```
printf("%5d", 42);    // Output: "   42" (right-aligned)
printf("%-5d", 42);   // Output: "42   " (left-aligned)
```

2. Precision:

```
printf("%.2f", 3.14159); // Output: "3.14"
printf("%.4s", "Hello");  // Output: "Hell" (first 4 characters)
```

3. Combining Width and Precision:

```
printf("%7.2f", 3.14159); // Output: "   3.14" (7 characters wide)
```

4. Zero Padding:

```
printf("%05d", 42); // Output: "00042"
```

Other Output Functions

1. puts() : Outputs a string followed by a newline

```
puts("Hello, world!"); // Simpler than printf(), automatically adds newline
```

2. putchar() : Outputs a single character

```
putchar('A'); // Outputs the character 'A'
```

3. fprintf() : Prints formatted output to a file


```
FILE *file = fopen("output.txt", "w");
fprintf(file, "Value: %d\n", 42);
fclose(file);
```

Example: Formatted Table Output

```
#include <stdio.h>

int main() {
    printf("%-15s %-10s %-10s\n", "Name", "Age", "Height");
    printf("%-15s %-10d %-10.2f\n", "John Smith", 25, 5.11);
    printf("%-15s %-10d %-10.2f\n", "Jane Doe", 28, 5.6);
    printf("%-15s %-10d %-10.2f\n", "Bob Johnson", 32, 6.2);

    return 0;
}
```

Output:

Name	Age	Height
John Smith	25	5.11
Jane Doe	28	5.60
Bob Johnson	32	6.20

Variables and Data Types

Variables are named storage locations in memory that hold values which can be modified during program execution. C requires all variables to be declared with their data type before they can be used.

Variable Declaration and Initialization

```
// Declaration
data_type variable_name;

// Declaration with initialization
data_type variable_name = initial_value;

// Multiple declarations
data_type variable1, variable2, variable3;
```

Example:

```
#include <stdio.h>

int main() {
    // Declaration
    int age;

    // Initialization after declaration
    age = 25;

    // Declaration with initialization
    float salary = 50000.50;

    // Multiple variables
    char grade = 'A', section = 'B';

    printf("Age: %d\n", age);
    printf("Salary: $%.2f\n", salary);
    printf("Grade: %c, Section: %c\n", grade, section);

    return 0;
}
```

Fundamental Data Types in C

Data Type	Description	Format Specifier	Typical Size	Example
int	Integer numbers	%d , %i	4 bytes	int count = 42;
char	Single character	%c	1 byte	char grade = 'A';
float	Single-precision floating-point	%f	4 bytes	float price = 9.99;
double	Double-precision floating-point	%lf	8 bytes	double pi = 3.14159265359;
_Bool	Boolean value (0 or 1)	%d	1 byte	_Bool flag = 1;

Type Modifiers

C provides modifiers to alter the range or precision of basic data types:

- 1. **Size Modifiers:**
 - o short - smaller integer (typically 2 bytes)
 - o long - larger integer (typically 4 or 8 bytes)
 - o long long - even larger integer (typically 8 bytes)
- 2. **Sign Modifiers:**

- `signed` - can represent negative and positive numbers (default)
- `unsigned` - can only represent non-negative numbers

Examples:

```
unsigned int positive_number = 50000;
long int large_number = 2000000000;
short int small_number = 32000;
unsigned long int very_large_positive = 4000000000;
```

Memory Sizes and Ranges

The exact size of data types can vary by platform. The `sizeof` operator can be used to determine the size:

```
#include <stdio.h>

int main() {
    printf("Size of char: %zu bytes\n", sizeof(char));
    printf("Size of int: %zu bytes\n", sizeof(int));
    printf("Size of float: %zu bytes\n", sizeof(float));
    printf("Size of double: %zu bytes\n", sizeof(double));

    return 0;
}
```

Typical ranges:

- `char` : -128 to 127 or 0 to 255 (unsigned)
- `int` : -2,147,483,648 to 2,147,483,647
- `unsigned int` : 0 to 4,294,967,295
- `float` : 1.2E-38 to 3.4E+38 (6-7 significant digits)
- `double` : 2.3E-308 to 1.7E+308 (15-16 significant digits)

Derived Data Types

Arrays

Arrays store multiple items of the same type:

```
int scores[5] = {95, 88, 76, 90, 82}; // Array of 5 integers
char name[10] = "John";               // Character array (string)

// Accessing array elements (zero-indexed)
```

```
printf("First score: %d\n", scores[0]); // Output: 95
printf("Name: %s\n", name);           // Output: John
```

Derived from Basic Types

1. Arrays: Multiple elements of same type

```
int numbers[5] = {1, 2, 3, 4, 5};
```

2. Pointers: Variables that store memory addresses

```
int *ptr;        // Pointer to an integer
int num = 10;
ptr = &num;      // Store address of num in ptr
```

3. Structures: Group related variables of different types

```
struct Student {
    char name[50];
    int age;
    float gpa;
};

struct Student student1 = {"Alice", 20, 3.8};
```

4. Unions: Store different data types in the same memory location

```
union Data {
    int i;
    float f;
    char str[20];
};

union Data data;
data.i = 10; // Data now holds an integer
```

5. Enumerations: User-defined type consisting of integral constants

```
enum Days {MON, TUE, WED, THU, FRI, SAT, SUN};
enum Days today = WED;
```

Type Conversion

C allows conversion between different data types:

1. Implicit Conversion (Automatic):

```
int i = 10;
float f = i;    // int converted to float (f = 10.0)
```

2. Explicit Conversion (Casting):

```
float f = 3.14;
int i = (int)f;  // Explicitly convert float to int (i = 3)
```

Constants

Constants are values that cannot be modified after declaration:

1. Using `const` qualifier:

```
const float PI = 3.14159;
```

2. Using `#define` directive:

```
#define MAX_SIZE 100
```

Example: Using Various Data Types

```
#include <stdio.h>

int main() {
    // Basic types
    int students = 30;
    float average_score = 85.6;
    char section = 'B';
    double exact_pi = 3.1415926535897932;

    // Modified types
    unsigned int total_population = 8000000;
    long int national_debt = 21000000000;

    // Array
    int top_scores[3] = {98, 97, 95};

    // Constant
    const int DAYS_IN_WEEK = 7;

    // Output
    printf("Class %c has %d students with average score %.1f\n",
           section, students, average_score);
    printf("Pi to 15 decimal places: %.15f\n", exact_pi);
    printf("Country population: %u\n", total_population);
    printf("National debt: %ld\n", national_debt);
    printf("Top score: %d\n", top_scores[0]);
}
```

```
printf("Days in a week: %d (this cannot be changed)\n", DAYS_IN_WEEK);

return 0;
}
```

Input Functions, Range of Values, and Qualifiers

Input functions allow programs to receive data from users. The primary input functions in C are defined in the `stdio.h` header.

Basic Input Functions

1. scanf() Function

The `scanf()` function reads formatted input from the standard input (keyboard):

```
#include <stdio.h>

int main() {
    int age;
    printf("Enter your age: ");
    scanf("%d", &age); // Note the & operator to get the address

    printf("You entered: %d\n", age);
    return 0;
}
```

Format Specifiers for scanf()

The format specifiers used with `scanf()` are similar to those used with `printf()` :

Specifier	Data Type	Example
%d , %i	Integer	scanf("%d", &num);
%f	Float	scanf("%f", &price);
%lf	Double	scanf("%lf", &pi);
%c	Character	scanf("%c", &letter);
%s	String	scanf("%s", name);
%x , %X	Hexadecimal	scanf("%x", &hex_value);
%o	Octal	scanf("%o", &oct_value);
%u	Unsigned integer	scanf("%u", &u_num);

Reading Multiple Values

You can read multiple values in a single `scanf()` call:

```
#include <stdio.h>

int main() {
    int id;
    float salary;

    printf("Enter ID and salary: ");
    scanf("%d %f", &id, &salary);

    printf("ID: %d, Salary: $%.2f\n", id, salary);
    return 0;
}
```

Common Issues with `scanf()`

1. **Buffer Issues:** `scanf()` leaves the newline character in the input buffer, which can affect subsequent reads
2. **String Input:** `scanf("%s", str)` stops at whitespace
3. **Format Mismatches:** Entering wrong data type causes unpredictable behavior

Example showing buffer problem:

```
#include <stdio.h>

int main() {
    int age;
    char name[50];

    printf("Enter age: ");
    scanf("%d", &age);

    // The newline from previous input remains in buffer
    printf("Enter name: ");
    scanf("%s", name);

    printf("Age: %d, Name: %s\n", age, name);
    return 0;
}
```

Solution:

```
#include <stdio.h>

int main() {
```

```

int age;
char name[50];

printf("Enter age: ");
scanf("%d", &age);

// Clear input buffer
while (getchar() != '\n');

printf("Enter name: ");
scanf("%s", name);

printf("Age: %d, Name: %s\n", age, name);
return 0;
}

```

Range of Values for Different Data Types

Understanding the range of values each data type can hold helps prevent overflow errors:

Data Type	Typical Range	Min Value	Max Value
char	-128 to 127	CHAR_MIN	CHAR_MAX
unsigned char	0 to 255	0	UCHAR_MAX
int	-2,147,483,648 to 2,147,483,647	INT_MIN	INT_MAX
unsigned int	0 to 4,294,967,295	0	UINT_MAX
short int	-32,768 to 32,767	SHRT_MIN	SHRT_MAX
unsigned short	0 to 65,535	0	USHRT_MAX
long int	-2,147,483,648 to 2,147,483,647 (32-bit)	LONG_MIN	LONG_MAX
	-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807 (64-bit)		
unsigned long	0 to 4,294,967,295 (32-bit)	0	ULONG_MAX
	0 to 18,446,744,073,709,551,615 (64-bit)		
float	~1.2E-38 to 3.4E+38 with ~7 significant digits	FLT_MIN	FLT_MAX
double	~2.3E-308 to 1.7E+308 with ~15 significant digits	DBL_MIN	DBL_MAX

To access these limits, include the `limits.h` and `float.h` headers:

```
#include <stdio.h>
#include <limits.h>
#include <float.h>

int main() {
    printf("Range of int: %d to %d\n", INT_MIN, INT_MAX);
    printf("Range of unsigned int: 0 to %u\n", UINT_MAX);
    printf("Range of float: %e to %e\n", FLT_MIN, FLT_MAX);
    printf("Float precision: %d digits\n", FLT_DIG);

    return 0;
}
```

Type Qualifiers

Type qualifiers modify the behavior of variables:

1. `const` Qualifier

Makes a variable immutable (read-only):

```
const int MAX_USERS = 100;
// MAX_USERS = 200; // Error: assignment of read-only variable
```

2. `volatile` Qualifier

Tells the compiler that a variable's value may change unexpectedly:

```
volatile int sensor_reading; // Value might change due to hardware
```

3. `restrict` Qualifier (C99 and later)

Informs the compiler that a pointer is the only means to access an object:

```
void copy(int * restrict dest, int * restrict src, int size) {
    for (int i = 0; i < size; i++) {
        dest[i] = src[i];
    }
}
```

Advanced Input Techniques

1. Reading Strings with Spaces

```
#include <stdio.h>

int main() {
    char full_name[100];

    printf("Enter your full name: ");
    // %[^\n] reads until newline
    scanf("%[^\n]", full_name);

    printf("Hello, %s!\n", full_name);
    return 0;
}
```

2. Using `fgets()` for Safer String Input

```
#include <stdio.h>

int main() {
    char address[100];

    printf("Enter your address: ");
    fgets(address, 100, stdin);

    printf("Your address is: %s", address);
    return 0;
}
```

3. Input Validation Example

```
#include <stdio.h>

int main() {
    int age;
    int valid_input = 0;

    do {
        printf("Enter your age (1-120): ");
        if (scanf("%d", &age) == 1) {
            if (age >= 1 && age <= 120) {
                valid_input = 1;
            } else {
                printf("Age must be between 1 and 120.\n");
            }
        } else {
            printf("Invalid input. Please enter a number.\n");
            while (getchar() != '\n'); // Clear input buffer
        }
    } while (!valid_input);
}
```

```

    printf("Your age is: %d\n", age);
    return 0;
}

```

Example: Using Input, Types, and Ranges Together

The following program reads student information and validates the input based on acceptable ranges:

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

// Constants (qualifiers example)
const int MIN_AGE = 16;
const int MAX_AGE = 60;
const float MIN_GPA = 0.0;
const float MAX_GPA = 4.0;

int main() {
    char name[50];
    int age;
    float gpa;
    int valid_input = 0;

    // Get and validate name
    printf("Enter student name: ");
    fgets(name, 50, stdin);

    // Remove trailing newline from fgets
    name[strcspn(name, "\n")] = 0;

    // Get and validate age
    do {
        printf("Enter student age (%d-%d): ", MIN_AGE, MAX_AGE);
        if (scanf("%d", &age) == 1) {
            if (age >= MIN_AGE && age <= MAX_AGE) {
                valid_input = 1;
            } else {
                printf("Age must be between %d and %d.\n", MIN_AGE, MAX_AGE);
            }
        } else {
            printf("Invalid input. Please enter a number.\n");
            while (getchar() != '\n'); // Clear input buffer
        }
    } while (!valid_input);

    // Clear input buffer
    while (getchar() != '\n');

    // Reset valid_input flag

```

```

    valid_input = 0;

    // Get and validate GPA
    do {
        printf("Enter student GPA (0.1f-1.0f): ", MIN_GPA, MAX_GPA);
        if (scanf("%f", &gpa) == 1) {
            if (gpa >= MIN_GPA && gpa <= MAX_GPA) {
                valid_input = 1;
            } else {
                printf("GPA must be between 0.1f and 1.0f.\n", MIN_GPA, MAX_GPA);
            }
        } else {
            printf("Invalid input. Please enter a number.\n");
            while (getchar() != '\n'); // Clear input buffer
        }
    } while (!valid_input);

    // Display student information
    printf("\nStudent Information:\n");
    printf("-----\n");
    printf("Name: %s\n", name);
    printf("Age: %d\n", age);
    printf("GPA: %.2f\n", gpa);

    return 0;
}

```

This program demonstrates:

- Using constants (`const` qualifier)
- Reading different types of input
- Validating input based on acceptable ranges
- Handling input errors
- Clearing the input buffer to prevent issues

Operators and Expressions

Operators are symbols that tell the compiler to perform specific operations.

Operators and Expressions (Continued)

Operators in C allow you to manipulate variables and values through various operations. Understanding them is crucial for effective programming.

Arithmetic Operators

Operator	Description	Example
+	Addition	a + b
-	Subtraction	a - b
*	Multiplication	a * b
/	Division	a / b
%	Modulus (remainder)	a % b
++	Increment	a++ or ++a
--	Decrement	a-- or --a

Examples:

```
#include <stdio.h>

int main() {
    int a = 10, b = 3;

    printf("Addition: %d + %d = %d\n", a, b, a + b);           // 13
    printf("Subtraction: %d - %d = %d\n", a, b, a - b);       // 7
    printf("Multiplication: %d * %d = %d\n", a, b, a * b);     // 30
    printf("Division: %d / %d = %d\n", a, b, a / b);           // 3 (integer division)
    printf("Modulus: %d %% %d = %d\n", a, b, a % b);           // 1

    int c = a++; // Post-increment: c = 10, a = 11
    printf("Post-increment: c = %d, a = %d\n", c, a);

    int d = ++b; // Pre-increment: d = 4, b = 4
    printf("Pre-increment: d = %d, b = %d\n", d, b);

    return 0;
}
```

Integer vs Floating-point Division

```
#include <stdio.h>

int main() {
    int a = 5, b = 2;
    float c = 5.0, d = 2.0;

    printf("Integer division: %d / %d = %d\n", a, b, a / b);           // 2
    printf("Float division: %.1f / %.1f = %.1f\n", c, d, c / d);       // 2.5
    printf("Mixed division: %d / %.1f = %.1f\n", a, d, a / d);         // 2.5
}
```

```
// Type casting for division
printf("Cast division: (float)%d / %d = %.1f\n", a, b, (float)a / b); // 2.5

return 0;
}
```

Relational Operators

Relational operators compare values and return a boolean result (1 for true, 0 for false).

Operator	Description	Example
==	Equal to	a == b
!=	Not equal to	a != b
>	Greater than	a > b
<	Less than	a < b
>=	Greater than or equal to	a >= b
<=	Less than or equal to	a <= b

Example:

```
#include <stdio.h>

int main() {
    int a = 10, b = 20;

    printf("%d == %d is %d\n", a, b, a == b); // 0 (false)
    printf("%d != %d is %d\n", a, b, a != b); // 1 (true)
    printf("%d > %d is %d\n", a, b, a > b);    // 0 (false)
    printf("%d < %d is %d\n", a, b, a < b);    // 1 (true)
    printf("%d >= %d is %d\n", a, b, a >= b); // 0 (false)
    printf("%d <= %d is %d\n", a, b, a <= b); // 1 (true)

    return 0;
}
```

Logical Operators

Logical operators combine boolean values.

Operator	Description	Example
&&	Logical AND	a && b

Operator	Description	Example
\	\	`
!	Logical NOT	!a

Example:

```
#include <stdio.h>

int main() {
    int a = 1, b = 0; // In C, 0 is false, non-zero is true

    printf("%d && %d is %d\n", a, b, a && b); // 0 (false)
    printf("%d || %d is %d\n", a, b, a || b); // 1 (true)
    printf("!!d is %d\n", a, !a); // 0 (false)
    printf("!!d is %d\n", b, !b); // 1 (true)

    // Short-circuit evaluation
    int x = 5, y = 0;
    // Second part not evaluated if first part is false (AND)
    if (y != 0 && x/y > 2) {
        printf("This won't execute\n");
    }

    // Second part not evaluated if first part is true (OR)
    if (x > 0 || x/y > 2) {
        printf("This will execute safely\n");
    }

    return 0;
}
```

Bitwise Operators

Bitwise operators manipulate individual bits in integer values.

Operator	Description	Example
&	Bitwise AND	a & b
\	`	Bitwise OR
^	Bitwise XOR	a ^ b
~	Bitwise NOT	~a
<<	Left shift	a << n
>>	Right shift	a >> n

Example:

```
#include <stdio.h>

int main() {
    unsigned int a = 60; // 00111100 in binary
    unsigned int b = 13; // 00001101 in binary

    printf("a = %u (binary: %08x)\n", a, a);
    printf("b = %u (binary: %08x)\n", b, b);

    printf("a & b = %u\n", a & b); // 00001100 = 12
    printf("a | b = %u\n", a | b); // 00111101 = 61
    printf("a ^ b = %u\n", a ^ b); // 00110001 = 49
    printf("~a = %u\n", ~a); // 11000011... = 4294967235 (32-bit)
    printf("a << 2 = %u\n", a << 2); // 11110000 = 240
    printf("a >> 2 = %u\n", a >> 2); // 00001111 = 15

    return 0;
}
```

Bitwise operators are useful for:

- 1. Flag manipulation (setting/clearing specific bits)
- 2. Resource-efficient storage of boolean values
- 3. Low-level hardware manipulation
- 4. Optimizing certain mathematical operations

Assignment Operators

Assignment operators assign values to variables, often combining assignment with another operation.

Operator	Description	Equivalent to
=	Simple assignment	-
+=	Add and assign	a = a + b
-=	Subtract and assign	a = a - b
*=	Multiply and assign	a = a * b
/=	Divide and assign	a = a / b
%=	Modulus and assign	a = a % b
&=	Bitwise AND and assign	a = a & b
\	=`	Bitwise OR and assign

Operator	Description	Equivalent to
<code>^=</code>	Bitwise XOR and assign	<code>a = a ^ b</code>
<code><<=</code>	Left shift and assign	<code>a = a << b</code>
<code>>>=</code>	Right shift and assign	<code>a = a >> b</code>

Example:

```
#include <stdio.h>

int main() {
    int a = 10;

    a += 5; // a = a + 5
    printf("After a += 5: a = %d\n", a); // 15

    a -= 3; // a = a - 3
    printf("After a -= 3: a = %d\n", a); // 12

    a *= 2; // a = a * 2
    printf("After a *= 2: a = %d\n", a); // 24

    a /= 4; // a = a / 4
    printf("After a /= 4: a = %d\n", a); // 6

    a %= 4; // a = a % 4
    printf("After a %= 4: a = %d\n", a); // 2

    return 0;
}
```

Conditional (Ternary) Operator

The conditional operator `?:` provides a shorthand way to write simple if-else statements.

Syntax: `condition ? expression_if_true : expression_if_false`

Example:

```
#include <stdio.h>

int main() {
    int age = 20;

    // Using if-else
    if (age >= 18) {
        printf("Adult\n");
    }
}
```

```

    } else {
        printf("Minor\n");
    }

    // Using ternary operator
    printf("%s\n", age >= 18 ? "Adult" : "Minor");

    // Assigning values based on condition
    int ticket_price = age < 12 ? 5 : (age < 65 ? 10 : 7);
    printf("Ticket price: $%d\n", ticket_price);

    return 0;
}

```

Special Operators

1. Sizeof Operator

Returns the size of a variable or data type in bytes.

```

#include <stdio.h>

int main() {
    int a;
    float b;
    double c;
    char d;

    printf("Size of int: %zu bytes\n", sizeof(a));    // Typically 4
    printf("Size of float: %zu bytes\n", sizeof(b));  // Typically 4
    printf("Size of double: %zu bytes\n", sizeof(c)); // Typically 8
    printf("Size of char: %zu bytes\n", sizeof(d));   // 1
    printf("Size of array: %zu bytes\n", sizeof(int[10])); // Typically 40

    return 0;
}

```

2. Address (&) and Dereference (*) Operators

The address operator (&) returns the memory address of a variable. The dereference operator (*) accesses the value at a memory address.

```

#include <stdio.h>

int main() {
    int x = 10;
    int *ptr;

    ptr = &x; // ptr now holds the address of x
}

```

```

printf("Value of x: %d\n", x);           // 10
printf("Address of x: %p\n", (void*)&x); // Memory address (e.g., 0x7ffffeddb0dc4c)
printf("Value of ptr: %p\n", (void*)ptr); // Same address as &x
printf("Value *ptr points to: %d\n", *ptr); // 10

*ptr = 20; // Change the value at the address ptr points to
printf("After *ptr = 20, x = %d\n", x);   // 20

return 0;
}

```

3. Structure Member Operators (. and ->)

Access members of structures and unions.

```

#include <stdio.h>

struct Person {
    char name[50];
    int age;
};

int main() {
    struct Person person1 = {"John", 25};
    struct Person *ptr = &person1;

    // Dot operator for direct access
    printf("Name: %s, Age: %d\n", person1.name, person1.age);

    // Arrow operator for pointer access
    printf("Name: %s, Age: %d\n", ptr->name, ptr->age);

    // Equivalent way using dereference and dot
    printf("Name: %s, Age: %d\n", (*ptr).name, (*ptr).age);

    return 0;
}

```

Operator Precedence and Associativity

Operator precedence determines the order in which operations are performed. Higher precedence operators are evaluated first.

Here's a simplified precedence table (from highest to lowest):

1. Postfix operators: `()` , `[]` , `->` , `.` , `++` (postfix), `--` (postfix)
2. Prefix operators: `++` (prefix), `--` (prefix), `+` (unary), `-` (unary), `!` , `~` , `(type)` , `*` , `&` , `sizeof`
3. Multiplicative: `*` , `/` , `%`

4. Additive: + , -
5. Shift: << , >>
6. Relational: < , <= , > , >=
7. Equality: == , !=
8. Bitwise AND: &
9. Bitwise XOR: ^
10. Bitwise OR: |
11. Logical AND: &&
12. Logical OR: ||
13. Conditional: ?:
14. Assignment: = , += , -= , etc.
15. Comma: ,

Example demonstrating precedence:

```
#include <stdio.h>

int main() {
    int a = 5, b = 3, c = 8, result;

    // Without parentheses (following precedence rules)
    result = a + b * c;
    printf("a + b * c = %d\n", result); // 5 + (3 * 8) = 29

    // With parentheses (changing the order)
    result = (a + b) * c;
    printf("(a + b) * c = %d\n", result); // (5 + 3) * 8 = 64

    // Complex expression
    result = a * b + c / 2 - 1;
    printf("a * b + c / 2 - 1 = %d\n", result); // (5 * 3) + (8 / 2) - 1 = 15 + 4 - 1 = 18

    return 0;
}
```

Common Expressions and Their Uses

1. Arithmetic Expressions

```
#include <stdio.h>
#include <math.h> // For mathematical functions

int main() {
    // Area of a circle
    double radius = 5.0;
```

```

double area = M_PI * radius * radius; // or M_PI * pow(radius, 2)
printf("Area of circle: %.2f\n", area); // 78.54

// Temperature conversion
double celsius = 25.0;
double fahrenheit = (celsius * 9.0/5.0) + 32.0;
printf("%.1f°C = %.1f°F\n", celsius, fahrenheit); // 25.0°C = 77.0°F

// Average of values
int values[] = {12, 45, 67, 23, 9};
int sum = 0;
for (int i = 0; i < 5; i++) {
    sum += values[i];
}
double average = (double)sum / 5;
printf("Average: %.2f\n", average); // 31.20

return 0;
}

```

2. Logical Expressions

```

#include <stdio.h>

int main() {
    int age = 25;
    float gpa = 3.5;

    // Checking multiple conditions
    if (age >= 18 && age <= 30) {
        printf("Young adult\n");
    }

    // Checking inclusive ranges
    if (gpa >= 3.0 && gpa <= 4.0) {
        printf("Good academic standing\n");
    }

    // Checking exclusive conditions
    char grade = 'B';
    if (grade == 'A' || grade == 'B') {
        printf("Above average performance\n");
    }

    // Negating conditions
    int is_weekend = 0; // 0 = false, 1 = true
    if (!is_weekend) {
        printf("It's a weekday\n");
    }
}

```

```
    return 0;
}
```

3. Bitwise Manipulation Expressions

```
#include <stdio.h>

int main() {
    // Setting a bit
    unsigned char flags = 0; // 00000000
    flags |= (1 << 2);        // Set bit 2: 00000100
    printf("After setting bit 2: %d\n", flags); // 4

    // Clearing a bit
    flags &= ~(1 << 2);        // Clear bit 2: 00000000
    printf("After clearing bit 2: %d\n", flags); // 0

    // Toggling a bit
    flags ^= (1 << 3);        // Toggle bit 3: 00001000
    printf("After toggling bit 3: %d\n", flags); // 8

    // Checking if a bit is set
    flags = 12; // 00001100
    if (flags & (1 << 2)) {
        printf("Bit 2 is set\n"); // Will print
    }

    if (flags & (1 << 0)) {
        printf("Bit 0 is set\n"); // Won't print
    }

    return 0;
}
```

Example: Calculator Program Using Various Operators

```
#include <stdio.h>
#include <math.h>

int main() {
    double num1, num2, result;
    char operator;

    printf("Enter a simple expression (e.g., 5 + 3): ");
    scanf("%lf %c %lf", &num1, &operator, &num2);

    switch (operator) {
        case '+':
            result = num1 + num2;
```

```

        printf("%.2f + %.2f = %.2f\n", num1, num2, result);
        break;
    case '-':
        result = num1 - num2;
        printf("%.2f - %.2f = %.2f\n", num1, num2, result);
        break;
    case '*':
        result = num1 * num2;
        printf("%.2f * %.2f = %.2f\n", num1, num2, result);
        break;
    case '/':
        if (num2 != 0) {
            result = num1 / num2;
            printf("%.2f / %.2f = %.2f\n", num1, num2, result);
        } else {
            printf("Error: Division by zero\n");
        }
        break;
    case '%':
        if (num2 != 0) {
            result = fmod(num1, num2); // Use fmod for floating-point modulus
            printf("%.2f %% %.2f = %.2f\n", num1, num2, result);
        } else {
            printf("Error: Division by zero\n");
        }
        break;
    case '^':
        result = pow(num1, num2);
        printf("%.2f ^ %.2f = %.2f\n", num1, num2, result);
        break;
    default:
        printf("Error: Unsupported operator\n");
}

return 0;
}

```

Programming using Variables, Data Types, Operators, and I/O Functions

Let's put all the concepts learned so far into practice with some comprehensive examples:

Example 1: Temperature Conversion Program

This program converts temperatures between Celsius and Fahrenheit:

```

#include <stdio.h>

int main() {

```

```

float celsius, fahrenheit;
char choice;

printf("Temperature Conversion Program\n");
printf("-----\n");
printf("Enter 'C' to convert Fahrenheit to Celsius\n");
printf("Enter 'F' to convert Celsius to Fahrenheit\n");
printf("Your choice: ");
scanf(" %c", &choice);

switch (choice) {
    case 'C':
    case 'c':
        printf("Enter temperature in Fahrenheit: ");
        scanf("%f", &fahrenheit);
        celsius = (fahrenheit - 32) * 5 / 9;
        printf("%.2f°F = %.2f°C\n", fahrenheit, celsius);
        break;

    case 'F':
    case 'f':
        printf("Enter temperature in Celsius: ");
        scanf("%f", &celsius);
        fahrenheit = (celsius * 9 / 5) + 32;
        printf("%.2f°C = %.2f°F\n", celsius, fahrenheit);
        break;

    default:
        printf("Invalid choice!\n");
}

return 0;
}

```

Example 2: Simple Banking System

This program demonstrates a simple bank account system:

```

#include <stdio.h>

int main() {
    double balance = 1000.0; // Initial balance
    int choice;
    double amount;

    printf("Simple Banking System\n");
    printf("-----\n");
    printf("Current Balance: $%.2f\n\n", balance);

    do {
        printf("1. Deposit\n");

```



```

printf("2. Withdraw\n");
printf("3. Check Balance\n");
printf("4. Exit\n");
printf("Enter your choice (1-4): ");
scanf("%d", &choice);

switch (choice) {
    case 1: // Deposit
        printf("Enter deposit amount: $");
        scanf("%lf", &amount);

        if (amount > 0) {
            balance += amount;
            printf("%.2f deposited successfully.\n", amount);
        } else {
            printf("Invalid amount. Please enter a positive value.\n");
        }
        break;

    case 2: // Withdraw
        printf("Enter withdrawal amount: $");
        scanf("%lf", &amount);

        if (amount > 0) {
            if (amount <= balance) {
                balance -= amount;
                printf("%.2f withdrawn successfully.\n", amount);
            } else {
                printf("Insufficient funds. Your balance is %.2f\n", balance);
            }
        } else {
            printf("Invalid amount. Please enter a positive value.\n");
        }
        break;

    case 3: // Check Balance
        printf("Current Balance: %.2f\n", balance);
        break;

    case 4: // Exit
        printf("Thank you for using our banking system!\n");
        break;

    default:
        printf("Invalid choice. Please select 1-4.\n");
}

printf("\n");
} while (choice != 4);

return 0;
}

```

Example 3: Student Grade Calculator

This program calculates and displays student grades:

```
#include <stdio.h>

int main() {
    const int MAX_SUBJECTS = 5;
    int marks[MAX_SUBJECTS];
    int total = 0;
    float average;
    char grade;

    printf("Student Grade Calculator\n");
    printf("-----\n");

    // Input marks for each subject
    for (int i = 0; i < MAX_SUBJECTS; i++) {
        do {
            printf("Enter marks for Subject %d (0-100): ", i + 1);
            scanf("%d", &marks[i]);

            if (marks[i] < 0 || marks[i] > 100) {
                printf("Invalid marks. Please enter between 0 and 100.\n");
            }
        } while (marks[i] < 0 || marks[i] > 100);

        total += marks[i];
    }

    // Calculate average
    average = (float)total / MAX_SUBJECTS;

    // Determine grade
    if (average >= 90) {
        grade = 'A';
    } else if (average >= 80) {
        grade = 'B';
    } else if (average >= 70) {
        grade = 'C';
    } else if (average >= 60) {
        grade = 'D';
    } else {
        grade = 'F';
    }

    // Display results
    printf("\nResults:\n");
    printf("-----\n");

    for (int i = 0; i < MAX_SUBJECTS; i++) {
        printf("Subject %d: %d\n", i + 1, marks[i]);
    }
}
```

```

}

printf("\nTotal Marks: %d out of %d\n", total, MAX_SUBJECTS * 100);
printf("Average: %.2f%%\n", average);
printf("Grade: %c\n", grade);

if (grade == 'F') {
    printf("Status: Failed\n");
} else {
    printf("Status: Passed\n");
}

return 0;
}

```

Example 4: Simple Calculator with Memory Functions

This calculator program demonstrates the use of memory variables and mathematical operations:

```

#include <stdio.h>
#include <math.h>

int main() {
    double num1, num2, result = 0;
    double memory = 0; // Memory variable
    char operator;
    int choice;

    printf("Advanced Calculator with Memory\n");
    printf("-----\n");

    do {
        printf("\nCurrent Result: %.2f\n", result);
        printf("Memory Value: %.2f\n\n", memory);

        printf("1. Perform calculation\n");
        printf("2. Store to memory\n");
        printf("3. Recall from memory\n");
        printf("4. Clear memory\n");
        printf("5. Clear result\n");
        printf("6. Exit\n");
        printf("Enter your choice (1-6): ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: // Calculation
                printf("Enter operation (e.g., 5 + 3, or 'r' for result): ");
                scanf(" %c", &operator);

                if (operator == 'r' || operator == 'R') {
                    num1 = result;

```

```

    } else {
        scanf("%lf", &num1);
    }

    scanf(" %c", &operator);
    scanf("%lf", &num2);

    switch (operator) {
        case '+':
            result = num1 + num2;
            break;
        case '-':
            result = num1 - num2;
            break;
        case '*':
            result = num1 * num2;
            break;
        case '/':
            if (num2 != 0) {
                result = num1 / num2;
            } else {
                printf("Error: Division by zero\n");
                continue;
            }
            break;
        case '^':
            result = pow(num1, num2);
            break;
        case '%':
            if (num2 != 0) {
                result = fmod(num1, num2);
            } else {
                printf("Error: Division by zero\n");
                continue;
            }
            break;
        default:
            printf("Error: Unsupported operator\n");
            continue;
    }

    printf("%.2f %c %.2f = %.2f\n", num1, operator, num2, result);
    break;

case 2: // Store to memory
    memory = result;
    printf("Value %.2f stored to memory\n", result);
    break;

case 3: // Recall from memory
    result = memory;
    printf("Value %.2f recalled from memory\n", memory);
    break;

```

```

        case 4: // Clear memory
            memory = 0;
            printf("Memory cleared\n");
            break;

        case 5: // Clear result
            result = 0;
            printf("Result cleared\n");
            break;

        case 6: // Exit
            printf("Exiting calculator...\n");
            break;

        default:
            printf("Invalid choice. Please select 1-6.\n");
    }
} while (choice != 6);

return 0;
}

```

Control Structures - Selection Statements and Looping

Control structures allow you to control the flow of execution in a program. They are divided into two main categories: selection (decision-making) and iteration (looping).

Selection Statements

1. if Statement

The `if` statement executes a block of code if a specified condition is true.

```

if (condition) {
    // Code to execute if condition is true
}

```

Example:

```

#include <stdio.h>

int main() {
    int age;

    printf("Enter your age: ");
    scanf("%d", &age);
}

```

```
    if (age >= 18) {
        printf("You are eligible to vote.\n");
    }

    return 0;
}
```

2. if-else Statement

The `if-else` statement executes one block of code if a condition is true and another block if it is false.

```
if (condition) {
    // Code to execute if condition is true
} else {
    // Code to execute if condition is false
}
```

Example:

```
#include <stdio.h>

int main() {
    int number;

    printf("Enter a number: ");
    scanf("%d", &number);

    if (number % 2 == 0) {
        printf("%d is an even number.\n", number);
    } else {
        printf("%d is an odd number.\n", number);
    }

    return 0;
}
```

3. if-else if-else Ladder

The `if-else if-else` ladder checks multiple conditions in sequence.

```
if (condition1) {
    // Code to execute if condition1 is true
} else if (condition2) {
    // Code to execute if condition1 is false and condition2 is true
} else if (condition3) {
    // Code to execute if condition1 and condition2 are false and condition3 is true
} else {
```

```
    // Code to execute if all conditions are false  
}
```

Example:

```
#include <stdio.h>  
  
int main() {  
    int score;  
  
    printf("Enter your exam score (0-100): ");  
    scanf("%d", &score);  
  
    if (score >= 90) {  
        printf("Grade: A\n");  
    } else if (score >= 80) {  
        printf("Grade: B\n");  
    } else if (score >= 70) {  
        printf("Grade: C\n");  
    } else if (score >= 60) {  
        printf("Grade: D\n");  
    }
```

C Programming: Complete Guide

Table of Contents

1. [Control Structures](#)
 - [Selection Structures](#)
 - [Looping Structures](#)
2. [Character Input/Output](#)
 - [Character I/O Functions](#)
 - [String I/O Functions](#)
 - [Formatted I/O](#)
3. [Coding Standards and Guidelines](#)
 - [Naming Conventions](#)
 - [Indentation and Formatting](#)
 - [Comments and Documentation](#)
 - [Error Handling](#)
4. [Arrays - 1D, Initialization and Traversal](#)
 - [Declaration and Initialization](#)
 - [Accessing and Modifying Elements](#)
 - [Array Traversal Techniques](#)

5. Arrays - 2D

- Declaration and Initialization
- Accessing and Modifying Elements
- Common 2D Array Operations

6. Pointers

- Pointer Basics
- Pointer Arithmetic
- Pointers and Arrays

7. Functions

- Function Declaration and Definition
- Parameter Passing
- Return Values

8. Arrays and Functions

- Passing Arrays to Functions
- Returning Arrays from Functions

Control Structures

Control structures in C programming determine the flow of program execution. They fall into two main categories: selection structures and looping structures.

Selection Structures

Selection structures allow a program to choose different execution paths based on conditions.

if Statement

The `if` statement executes a block of code if a specified condition is true.

```
#include <stdio.h>

int main() {
    int age = 20;

    if (age >= 18) {
        printf("You are an adult.\n");
    }

    return 0;
}
```

if-else Statement

The `if-else` statement provides an alternative path when the condition is false.


```
#include <stdio.h>

int main() {
    int number = 7;

    if (number % 2 == 0) {
        printf("%d is even.\n", number);
    } else {
        printf("%d is odd.\n", number);
    }

    return 0;
}
```

if-else if-else Ladder

For multiple conditions, the if-else if-else ladder can be used.

```
#include <stdio.h>

int main() {
    int score = 85;

    if (score >= 90) {
        printf("Grade: A\n");
    } else if (score >= 80) {
        printf("Grade: B\n");
    } else if (score >= 70) {
        printf("Grade: C\n");
    } else if (score >= 60) {
        printf("Grade: D\n");
    } else {
        printf("Grade: F\n");
    }

    return 0;
}
```

Nested if Statements

An if statement can be nested inside another if or else block.

```
#include <stdio.h>

int main() {
    int age = 25;
    int hasLicense = 1; // 1 for true, 0 for false
```

```

if (age >= 18) {
    printf("Age requirement met.\n");

    if (hasLicense) {
        printf("You can drive.\n");
    } else {
        printf("You need a license to drive.\n");
    }
} else {
    printf("You are too young to drive.\n");
}

return 0;
}

```

switch Statement

The `switch` statement provides a cleaner way to handle multiple conditions based on a single value.

```

#include <stdio.h>

int main() {
    char grade = 'B';

    switch (grade) {
        case 'A':
            printf("Excellent!\n");
            break;
        case 'B':
            printf("Good job!\n");
            break;
        case 'C':
            printf("Satisfactory.\n");
            break;
        case 'D':
            printf("Needs improvement.\n");
            break;
        case 'F':
            printf("Failed.\n");
            break;
        default:
            printf("Invalid grade.\n");
    }

    return 0;
}

```

Important notes about `switch` :

- Each `case` should end with a `break` statement to prevent fall-through.

- The `default` case handles any value not matched by the cases.
- The expression in the `switch` statement must be integral (`int`, `char`, `enum`).

Conditional Operator

The conditional operator (`? :`) provides a compact way to express simple if-else logic.

```
#include <stdio.h>

int main() {
    int age = 20;

    // Using conditional operator
    printf("You are %s.\n", (age >= 18) ? "an adult" : "a minor");

    // Equivalent if-else
    if (age >= 18) {
        printf("You are an adult.\n");
    } else {
        printf("You are a minor.\n");
    }

    return 0;
}
```

Looping Structures

Looping structures allow a program to repeat a block of code multiple times.

for Loop

The `for` loop is typically used when the number of iterations is known beforehand.

```
#include <stdio.h>

int main() {
    // Print numbers from 1 to 5
    for (int i = 1; i <= 5; i++) {
        printf("%d ", i);
    }
    printf("\n");

    // Calculate the sum of numbers from 1 to 10
    int sum = 0;
    for (int i = 1; i <= 10; i++) {
        sum += i;
    }
    printf("Sum: %d\n", sum);
}
```

```
    return 0;
}
```

The `for` loop has three parts:

1. **Initialization:** Executed once before the loop starts.
2. **Condition:** Checked before each iteration; if false, the loop terminates.
3. **Update:** Executed after each iteration.

while Loop

The `while` loop repeats as long as a condition is true.

```
#include <stdio.h>

int main() {
    int count = 1;

    while (count <= 5) {
        printf("%d ", count);
        count++;
    }
    printf("\n");

    // Reading until a specific input
    int number;
    printf("Enter a number (0 to quit): ");
    scanf("%d", &number);

    while (number != 0) {
        printf("You entered: %d\n", number);
        printf("Enter another number (0 to quit): ");
        scanf("%d", &number);
    }

    return 0;
}
```

do-while Loop

The `do-while` loop is similar to the `while` loop, but it guarantees that the loop body executes at least once.

```
#include <stdio.h>

int main() {
    int count = 1;
```

```

do {
    printf("%d ", count);
    count++;
} while (count <= 5);
printf("\n");

// Even if the condition is initially false, the loop executes once
int number = 10;

do {
    printf("This will print once even though number > 5\n");
} while (number <= 5);

return 0;
}

```

break and continue Statements

- **break** : Exits the innermost loop immediately.
- **continue** : Skips the rest of the current iteration and moves to the next iteration.

```

#include <stdio.h>

int main() {
    // Using break to exit the loop early
    printf("Break example:\n");
    for (int i = 1; i <= 10; i++) {
        if (i == 6) {
            break; // Exit the loop when i is 6
        }
        printf("%d ", i);
    }
    printf("\n");

    // Using continue to skip specific iterations
    printf("Continue example:\n");
    for (int i = 1; i <= 10; i++) {
        if (i % 2 == 0) {
            continue; // Skip even numbers
        }
        printf("%d ", i);
    }
    printf("\n");

    return 0;
}

```

Nested Loops

Loops can be nested within other loops to handle more complex patterns.

```
#include <stdio.h>

int main() {
    // Print a pattern
    for (int i = 1; i <= 5; i++) {
        for (int j = 1; j <= i; j++) {
            printf("* ");
        }
        printf("\n");
    }

    // Print a multiplication table
    printf("\nMultiplication Table:\n");
    for (int i = 1; i <= 5; i++) {
        for (int j = 1; j <= 5; j++) {
            printf("%d\t", i * j);
        }
        printf("\n");
    }

    return 0;
}
```

Character Input/Output

Character I/O operations allow a program to read and write characters and strings.

Character I/O Functions

getchar() and putchar()

`getchar()` reads a single character from `stdin`, and `putchar()` writes a single character to `stdout`.

```
#include <stdio.h>

int main() {
    printf("Enter a character: ");
    char ch = getchar();

    printf("You entered: ");
    putchar(ch);
    printf("\n");

    return 0;
}
```

fgetc() and fputc()

These functions work like `getchar()` and `putchar()` but allow specifying the stream.

```
#include <stdio.h>

int main() {
    // Reading from stdin (equivalent to getchar())
    printf("Enter a character: ");
    char ch = fgetc(stdin);

    // Writing to stdout (equivalent to putchar())
    printf("You entered: ");
    fputc(ch, stdout);
    printf("\n");

    // File I/O
    FILE *file = fopen("char.txt", "w");
    if (file != NULL) {
        fputc('A', file);
        fputc('B', file);
        fputc('C', file);
        fclose(file);

        printf("Characters written to file.\n");
    }

    return 0;
}
```

String I/O Functions

gets() and puts() (Deprecated)

`gets()` reads a line from stdin until a newline or EOF, and `puts()` writes a string to stdout followed by a newline.

```
#include <stdio.h>

int main() {
    char str[100];

    printf("Enter a string: ");
    // gets(str); // WARNING: gets() is deprecated due to buffer overflow risk

    // Use fgets() instead
    fgets(str, sizeof(str), stdin);

    printf("You entered: ");
    puts(str);
}
```

```
    return 0;
}
```

fgets() and fputs()

These functions provide safer string I/O and allow specifying the stream.

```
#include <stdio.h>

int main() {
    char str[100];

    printf("Enter a string: ");
    fgets(str, sizeof(str), stdin);

    // Remove the newline character if present
    for (int i = 0; str[i] != '\0'; i++) {
        if (str[i] == '\n') {
            str[i] = '\0';
            break;
        }
    }

    printf("You entered: ");
    fputs(str, stdout);
    printf("\n");

    // File I/O
    FILE *file = fopen("string.txt", "w");
    if (file != NULL) {
        fputs("Hello, World!", file);
        fclose(file);

        printf("String written to file.\n");
    }

    return 0;
}
```

Formatted I/O

printf() and scanf()

printf() formats and writes data to stdout, and scanf() reads formatted data from stdin.

```
#include <stdio.h>

int main() {
```



```

int age;
float height;
char name[50];

printf("Enter your name: ");
scanf("%49s", name); // Limit to 49 characters to avoid buffer overflow

printf("Enter your age and height (in meters): ");
scanf("%d %f", &age, &height);

printf("\nYour details:\n");
printf("Name: %s\n", name);
printf("Age: %d years\n", age);
printf("Height: %.2f meters\n", height);

return 0;
}

```

fprintf() and fscanf()

These functions work like `printf()` and `scanf()` but allow specifying the stream.

```

#include <stdio.h>

int main() {
    int id, age;
    char name[50];

    // Writing to a file
    FILE *file = fopen("data.txt", "w");
    if (file != NULL) {
        fprintf(file, "%d %s %d\n", 1, "John", 25);
        fprintf(file, "%d %s %d\n", 2, "Alice", 30);
        fclose(file);

        printf("Data written to file.\n");
    }

    // Reading from the file
    file = fopen("data.txt", "r");
    if (file != NULL) {
        printf("\nReading data from file:\n");

        while (fscanf(file, "%d %s %d", &id, name, &age) == 3) {
            printf("ID: %d, Name: %s, Age: %d\n", id, name, age);
        }

        fclose(file);
    }
}

```

```
    return 0;
}
```

sprintf() and sscanf()

These functions allow formatting and parsing strings in memory.

```
#include <stdio.h>

int main() {
    char buffer[100];
    int age = 25;
    float height = 1.75;

    // Format data into a string
    sprintf(buffer, "Age: %d, Height: %.2f meters", age, height);
    printf("Formatted string: %s\n", buffer);

    // Parse data from a string
    char str[] = "ID: 123 Score: 95.5";
    int id;
    float score;

    sscanf(str, "ID: %d Score: %f", &id, &score);
    printf("Parsed data - ID: %d, Score: %.1f\n", id, score);

    return 0;
}
```

Coding Standards and Guidelines

Coding standards ensure code readability, maintainability, and consistency across projects.

Naming Conventions

Variable and Function Names

- Use meaningful names that describe the purpose.
- Follow a consistent case style (e.g., camelCase or snake_case).

```
// Good practice
int studentCount;
float averageScore;
void calculateTotalSum();

// Bad practice
int a;
```

```
float x;  
void calc();
```

Constants and Macros

- Use uppercase letters with underscores for constants and macros.

```
#define MAX_BUFFER_SIZE 1024  
#define PI 3.14159  
  
const int MIN_AGE = 18;
```

Indentation and Formatting

Indentation

- Use consistent indentation (typically 2 or 4 spaces).
- Align braces properly.

```
// Consistent indentation example  
void processData(int value) {  
    if (value > 0) {  
        printf("Positive value\n");  
        value *= 2;  
    } else {  
        printf("Non-positive value\n");  
        value = 0;  
    }  
  
    printf("Final value: %d\n", value);  
}
```

Line Length

- Keep lines reasonably short (e.g., 80-120 characters).
- Break long lines logically.

```
// Long line broken into multiple lines  
int result = functionWithLongName(parameter1, parameter2,  
                                   parameter3, parameter4);  
  
// Long condition broken into multiple lines  
if (condition1 &&  
    condition2 &&  
    condition3) {
```

```
// Code block  
}
```

Comments and Documentation

Function Comments

- Document function purpose, parameters, and return values.

```
/**  
 * Calculates the average of an array of integers.  
 *  
 * @param arr The input array  
 * @param size The size of the array  
 * @return The average of the elements  
 */  
float calculateAverage(int arr[], int size) {  
    if (size <= 0) {  
        return 0.0f;  
    }  
  
    int sum = 0;  
    for (int i = 0; i < size; i++) {  
        sum += arr[i];  
    }  
  
    return (float)sum / size;  
}
```

Inline Comments

- Use inline comments to explain complex logic.

```
void processTransaction(Transaction *t) {  
    // Skip inactive transactions  
    if (!t->isActive) {  
        return;  
    }  
  
    // Apply discount for premium customers  
    if (t->customerType == PREMIUM) {  
        t->amount *= 0.9; // 10% discount  
    }  
  
    // Calculate tax based on region  
    calculateRegionalTax(t);  
}
```

Error Handling

Return Codes

- Use consistent return codes for error handling.

```
#define SUCCESS 0
#define ERROR_FILE_NOT_FOUND -1
#define ERROR_INVALID_INPUT -2

int processFile(const char *filename) {
    FILE *file = fopen(filename, "r");
    if (file == NULL) {
        return ERROR_FILE_NOT_FOUND;
    }

    // Process the file
    fclose(file);
    return SUCCESS;
}
```

Input Validation

- Always validate input before processing.

```
int divide(int a, int b, int *result) {
    if (b == 0) {
        return -1; // Error: division by zero
    }

    *result = a / b;
    return 0; // Success
}

int main() {
    int a = 10, b = 0, result;

    if (divide(a, b, &result) == 0) {
        printf("Result: %d\n", result);
    } else {
        printf("Error: Division by zero\n");
    }

    return 0;
}
```

Arrays - 1D, Initialization and Traversal

Arrays are collections of elements of the same type stored in contiguous memory locations.

Declaration and Initialization

Array Declaration

```
// Declaration without initialization
int numbers[5]; // Array of 5 integers

// Declaration with size determined by the compiler
int days[] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
```

Array Initialization

```
#include <stdio.h>

int main() {
    // Initialize at declaration
    int scores[5] = {95, 88, 76, 90, 82};

    // Partial initialization (remaining elements are set to 0)
    int values[5] = {10, 20}; // Equivalent to {10, 20, 0, 0, 0}

    // Initialize all elements to zero
    int zeros[10] = {0};

    // Initialize after declaration
    int grades[3];
    grades[0] = 85;
    grades[1] = 92;
    grades[2] = 78;

    return 0;
}
```

Accessing and Modifying Elements

```
#include <stdio.h>

int main() {
    int numbers[5] = {10, 20, 30, 40, 50};

    // Accessing elements
    printf("First element: %d\n", numbers[0]);
    printf("Third element: %d\n", numbers[2]);

    // Modifying elements
    numbers[1] = 25;
```

```

numbers[4] = 55;

// Displaying all elements
printf("Updated array: ");
for (int i = 0; i < 5; i++) {
    printf("%d ", numbers[i]);
}
printf("\n");

return 0;
}

```

Array Traversal Techniques

Using for Loop

```

#include <stdio.h>

int main() {
    int numbers[5] = {10, 20, 30, 40, 50};
    int sum = 0;

    // Simple traversal
    printf("Array elements: ");
    for (int i = 0; i < 5; i++) {
        printf("%d ", numbers[i]);
    }
    printf("\n");

    // Traversal with calculation
    for (int i = 0; i < 5; i++) {
        sum += numbers[i];
    }
    printf("Sum of elements: %d\n", sum);

    // Reverse traversal
    printf("Array in reverse: ");
    for (int i = 4; i >= 0; i--) {
        printf("%d ", numbers[i]);
    }
    printf("\n");

    return 0;
}

```

Using while Loop

```

#include <stdio.h>

```

```

int main() {
    int numbers[5] = {10, 20, 30, 40, 50};
    int i = 0;

    printf("Array elements (using while): ");
    while (i < 5) {
        printf("%d ", numbers[i]);
        i++;
    }
    printf("\n");

    return 0;
}

```

Using Pointers

```

#include <stdio.h>

int main() {
    int numbers[5] = {10, 20, 30, 40, 50};
    int *ptr = numbers; // Pointer to the first element

    printf("Array elements (using pointer): ");
    for (int i = 0; i < 5; i++) {
        printf("%d ", *(ptr + i));
    }
    printf("\n");

    // Alternative pointer traversal
    printf("Array elements (using pointer arithmetic): ");
    for (int *p = numbers; p < numbers + 5; p++) {
        printf("%d ", *p);
    }
    printf("\n");

    return 0;
}

```

Arrays - 2D

Two-dimensional arrays represent a table of elements with rows and columns.

2D Declaration and Initialization

```

#include <stdio.h>

int main() {
    // Declaration with initialization

```



```

int matrix[3][4] = {
    {1, 2, 3, 4},
    {5, 6, 7, 8},
    {9, 10, 11, 12}
};

// Partial initialization
int grid[2][3] = {
    {1, 2, 3}, // Row 0
    {4, 5}     // Row 1, column 2 defaults to 0
};

// Row-by-row initialization (same as matrix above)
int another[3][4] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12};

// Initialize all elements to zero
int zeros[3][3] = {{0}};

return 0;
}

```

2D Accessing and Modifying Elements

```

#include <stdio.h>

int main() {
    int matrix[3][3] = {
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 9}
    };

    // Accessing elements
    printf("Element at row 1, column 2: %d\n", matrix[1][2]); // Output: 6

    // Modifying elements
    matrix[0][0] = 10;
    matrix[2][2] = 99;

    // Display the modified matrix
    printf("Modified matrix:\n");
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            printf("%2d ", matrix[i][j]);
        }
        printf("\n");
    }

    return 0;
}

```

Common 2D Array Operations

Matrix Addition

```
#include <stdio.h>

int main() {
    int A[2][3] = {{1, 2, 3}, {4, 5, 6}};
    int B[2][3] = {{7, 8, 9}, {10, 11, 12}};
    int C[2][3]; // To store the result

    // Add matrices A and B
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 3; j++) {
            C[i][j] = A[i][j] + B[i][j];
        }
    }

    // Display the result
    printf("Matrix Addition Result:\n");
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 3; j++) {
            printf("%2d ", C[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

Matrix Multiplication

```
#include <stdio.h>

int main() {
    int A[2][3] = {{1, 2, 3}, {4, 5, 6}};
    int B[3][2] = {{7, 8}, {9, 10}, {11, 12}};
    int C[2][2] = {0}; // To store the result, initialized to zeros

    // Multiply matrices A and B
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            for (int k = 0; k < 3; k++) {
                C[i][j] += A[i][k] * B[k][j];
            }
        }
    }

    // Display the result
    printf("Matrix Multiplication Result:\n");
```

```

    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            printf("%3d ", C[i][j]);
        }
        printf("\n");
    }

    return 0;
}

```

Finding the Sum of Diagonals

```

#include <stdio.h>

int main() {
    int matrix[3][3] = {
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 9}
    };

    int mainDiagonalSum = 0;
    int antiDiagonalSum = 0;

    for (int i = 0; i < 3; i++) {
        mainDiagonalSum += matrix[i][i];           // Main diagonal
        antiDiagonalSum += matrix[i][3 - 1 - i];   // Anti-diagonal
    }

    printf("Main Diagonal Sum: %d\n", mainDiagonalSum);
    printf("Anti-Diagonal Sum: %d\n", antiDiagonalSum);

    return 0;
}

```

Pointers

Pointers are variables that store memory addresses.

Pointer Basics

```

#include <stdio.h>

int main() {
    int num = 10;
    int *ptr; // Declare a pointer to an integer

    ptr = &num; // Assign the address of num to ptr
}

```

```

printf("Value of num: %d\n", num);
printf("Address of num: %p\n", &num);
printf("Value of ptr (address it points to): %p\n", ptr);
printf("Value at the address stored in ptr: %d\n", *ptr);

// Modifying the value using pointer
*ptr = 20;
printf("New value of num: %d\n", num);

return 0;
}

```

Pointer Arithmetic

```

#include <stdio.h>

int main() {
    int numbers[5] = {10, 20, 30, 40, 50};
    int *ptr = numbers; // Points to the first element

    printf("Array traversal using pointer arithmetic:\n");

    // Accessing elements using pointer arithmetic
    printf("Element 0: %d\n", *ptr); // 10
    printf("Element 1: %d\n", *(ptr + 1)); // 20
    printf("Element 2: %d\n", *(ptr + 2)); // 30

    // Moving the pointer
    ptr++; // Now points to the second element
    printf("After ptr++, pointer points to: %d\n", *ptr); // 20

    ptr += 2; // Now points to the fourth element
    printf("After ptr += 2, pointer points to: %d\n", *ptr); // 40

    ptr--; // Now points back to the third element
    printf("After ptr--, pointer points to: %d\n", *ptr); // 30

    // Comparing pointers
    int *p1 = &numbers[1];
    int *p2 = &numbers[3];

    printf("Pointer comparison: p2 - p1 = %ld elements\n", p2 - p1);

    return 0;
}

```

Pointers and Arrays

```
#include <stdio.h>

int main() {
    int numbers[5] = {10, 20, 30, 40, 50};

    // Array name as a pointer
    printf("numbers is a pointer to the first element: %p\n", numbers);
    printf("&numbers[0] gives the same address: %p\n", &numbers[0]);

    // Equivalence of array indexing and pointer arithmetic
    printf("numbers[2] = %d\n", numbers[2]);
    printf("*(numbers + 2) = %d\n", *(numbers + 2));

    // These two expressions are equivalent
    printf("3[numbers] = %d\n", 3[numbers]);           // Unusual but valid
    printf("*(3 + numbers) = %d\n", *(3 + numbers));

    return 0;
}
```

Functions

Functions are self-contained blocks of code that perform specific tasks.

Function Declaration and Definition

```
#include <stdio.h>

// Function declaration (prototype)
int add(int a, int b);
void greet(char name[]);
float calculateAverage(int values[], int size);

int main() {
    int result = add(5, 3);
    printf("5 + 3 = %d\n", result);

    greet("John");

    int scores[] =
```

Functions (Continued)

```
#include <stdio.h>

// Function declaration (prototype)
```

```

int add(int a, int b);
void greet(char name[]);
float calculateAverage(int values[], int size);

int main() {
    int result = add(5, 3);
    printf("5 + 3 = %d\n", result);

    greet("John");

    int scores[] = {85, 90, 78, 92, 88};
    float avg = calculateAverage(scores, 5);
    printf("Average score: %.2f\n", avg);

    return 0;
}

// Function definition
int add(int a, int b) {
    return a + b;
}

void greet(char name[]) {
    printf("Hello, %s!\n", name);
}

float calculateAverage(int values[], int size) {
    if (size <= 0) return 0.0f;

    int sum = 0;
    for (int i = 0; i < size; i++) {
        sum += values[i];
    }

    return (float)sum / size;
}

```

Parameter Passing

Pass by Value

In C, all parameters are passed by value, meaning a copy of the argument is passed to the function.

```

#include <stdio.h>

void modifyValue(int x) {
    x = x * 2; // Modifies the local copy, not the original
    printf("Inside function: x = %d\n", x);
}

int main() {

```

```

int num = 10;

printf("Before function call: num = %d\n", num);
modifyValue(num);
printf("After function call: num = %d\n", num); // num remains unchanged

return 0;
}

```

Pass by Reference (using pointers)

To modify the original value, pass a pointer to the variable.

```

#include <stdio.h>

void modifyValue(int *x) {
    *x = *x * 2; // Modifies the original value
    printf("Inside function: *x = %d\n", *x);
}

int main() {
    int num = 10;

    printf("Before function call: num = %d\n", num);
    modifyValue(&num); // Pass the address of num
    printf("After function call: num = %d\n", num); // num is modified

    return 0;
}

```

Return Values

Functions can return a single value of any basic data type.

```

#include <stdio.h>

// Return an integer
int square(int x) {
    return x * x;
}

// Return a float
float celsiusToFahrenheit(float celsius) {
    return (celsius * 9.0f / 5.0f) + 32.0f;
}

// Return a character
char getGrade(int score) {
    if (score >= 90) return 'A';
}

```

```

    else if (score >= 80) return 'B';
    else if (score >= 70) return 'C';
    else if (score >= 60) return 'D';
    else return 'F';
}

int main() {
    int num = 5;
    float temp = 25.0f;
    int score = 85;

    printf("%d squared is %d\n", num, square(num));
    printf("%.1f°C is %.1f°F\n", temp, celsiusToFahrenheit(temp));
    printf("A score of %d gets a grade of %c\n", score, getGrade(score));

    return 0;
}

```

Arrays and Functions

Arrays have special behavior when passed to functions.

Passing Arrays to Functions

When an array is passed to a function, only the address of the first element is passed, not a copy of the entire array.

```

#include <stdio.h>

// Function that modifies an array
void modifyArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        arr[i] *= 2; // Modifies the original array
    }
}

// Function that processes an array without modifying it
void printArray(const int arr[], int size) {
    printf("Array elements: ");
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main() {
    int numbers[5] = {10, 20, 30, 40, 50};

    printf("Before modification:\n");

```



```

    printArray(numbers, 5);

    modifyArray(numbers, 5);

    printf("After modification:\n");
    printArray(numbers, 5);

    return 0;
}

```

Passing Multidimensional Arrays

For 2D arrays, the second (and subsequent) dimensions must be specified in the function parameter.

```

#include <stdio.h>

// Function to print a 2D array
void print2DArray(int arr[][3], int rows) {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < 3; j++) {
            printf("%d ", arr[i][j]);
        }
        printf("\n");
    }
}

// Function to find the sum of a 2D array
int sum2DArray(int arr[][3], int rows) {
    int total = 0;
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < 3; j++) {
            total += arr[i][j];
        }
    }
    return total;
}

int main() {
    int matrix[2][3] = {
        {1, 2, 3},
        {4, 5, 6}
    };

    printf("2D Array:\n");
    print2DArray(matrix, 2);

    int total = sum2DArray(matrix, 2);
    printf("Sum of all elements: %d\n", total);
}

```

```
    return 0;
}
```

Returning Arrays from Functions

C doesn't allow functions to directly return arrays, but there are several workarounds:

Return a Pointer to a Static Array

```
#include <stdio.h>

// Return a pointer to a static array (not recommended for general use)
int* generateNumbers() {
    static int numbers[5] = {10, 20, 30, 40, 50};
    return numbers;
}

int main() {
    int *result = generateNumbers();

    printf("Generated numbers: ");
    for (int i = 0; i < 5; i++) {
        printf("%d ", result[i]);
    }
    printf("\n");

    return 0;
}
```

Modify an Array Passed as Parameter

```
#include <stdio.h>

// Populate an array passed as parameter
void generateFibonacci(int arr[], int size) {
    if (size <= 0) return;

    arr[0] = 0;
    if (size == 1) return;

    arr[1] = 1;
    for (int i = 2; i < size; i++) {
        arr[i] = arr[i-1] + arr[i-2];
    }
}

int main() {
    int fibNumbers[10];
```

```

generateFibonacci(fibNumbers, 10);

printf("Fibonacci sequence: ");
for (int i = 0; i < 10; i++) {
    printf("%d ", fibNumbers[i]);
}
printf("\n");

return 0;
}

```

Use Dynamic Memory Allocation

```

#include <stdio.h>
#include <stdlib.h>

// Return a dynamically allocated array
int* createArray(int size, int value) {
    int *arr = (int*)malloc(size * sizeof(int));
    if (arr == NULL) {
        printf("Memory allocation failed!\n");
        exit(1);
    }

    for (int i = 0; i < size; i++) {
        arr[i] = value;
    }

    return arr;
}

int main() {
    int size = 5;
    int value = 7;

    int *result = createArray(size, value);

    printf("Created array: ");
    for (int i = 0; i < size; i++) {
        printf("%d ", result[i]);
    }
    printf("\n");

    // Don't forget to free the allocated memory
    free(result);

    return 0;
}

```

Array of Pointers

An array of pointers is an array where each element is a pointer.

Declaration and Initialization

```
#include <stdio.h>

int main() {
    // Array of integer pointers
    int a = 10, b = 20, c = 30;
    int *ptr_array[3];

    ptr_array[0] = &a;
    ptr_array[1] = &b;
    ptr_array[2] = &c;

    for (int i = 0; i < 3; i++) {
        printf("Value at ptr_array[%d]: %d\n", i, *ptr_array[i]);
    }

    // Array of string literals (char pointers)
    char *fruits[] = {
        "Apple",
        "Banana",
        "Cherry",
        "Orange"
    };

    printf("\nFruit list:\n");
    for (int i = 0; i < 4; i++) {
        printf("%d. %s\n", i+1, fruits[i]);
    }

    return 0;
}
```

Using Array of Pointers

Managing Variable-Length Strings

```
#include <stdio.h>
#include <string.h>

int main() {
    char *names[] = {
        "John",
        "Alice",
    }
```

```

    "Bob",
    "Catherine"
};

// Sort names alphabetically (simple bubble sort)
int n = 4;
for (int i = 0; i < n-1; i++) {
    for (int j = 0; j < n-i-1; j++) {
        if (strcmp(names[j], names[j+1]) > 0) {
            // Swap pointers
            char *temp = names[j];
            names[j] = names[j+1];
            names[j+1] = temp;
        }
    }
}

printf("Sorted names:\n");
for (int i = 0; i < n; i++) {
    printf("%s\n", names[i]);
}

return 0;
}

```

Array of Function Pointers

```

#include <stdio.h>

// Function declarations
int add(int a, int b) { return a + b; }
int subtract(int a, int b) { return a - b; }
int multiply(int a, int b) { return a * b; }
int divide(int a, int b) { return b != 0 ? a / b : 0; }

int main() {
    // Array of function pointers
    int (*operations[4])(int, int) = {add, subtract, multiply, divide};
    char *op_names[] = {"Addition", "Subtraction", "Multiplication", "Division"};

    int a = 10, b = 5;

    for (int i = 0; i < 4; i++) {
        printf("%s: %d\n", op_names[i], operations[i](a, b));
    }

    return 0;
}

```

Problem Solving using Arrays, Functions and Pointers

Let's explore some common problems that involve arrays, functions, and pointers.

Problem 1: Finding Maximum and Minimum in an Array

```
#include <stdio.h>

void findMinMax(int arr[], int size, int *min, int *max) {
    if (size <= 0) return;

    *min = *max = arr[0];

    for (int i = 1; i < size; i++) {
        if (arr[i] < *min) {
            *min = arr[i];
        }
        if (arr[i] > *max) {
            *max = arr[i];
        }
    }
}

int main() {
    int numbers[] = {34, 12, 89, 45, 23, 67, 5, 78};
    int size = sizeof(numbers) / sizeof(numbers[0]);
    int min, max;

    findMinMax(numbers, size, &min, &max);

    printf("Array: ");
    for (int i = 0; i < size; i++) {
        printf("%d ", numbers[i]);
    }
    printf("\n");

    printf("Minimum value: %d\n", min);
    printf("Maximum value: %d\n", max);

    return 0;
}
```

Problem 2: Matrix Transpose

```
#include <stdio.h>

// Function to transpose a matrix
void transposeMatrix(int original[][3], int transposed[][2], int rows, int cols) {
    for (int i = 0; i < rows; i++) {
```

```

        for (int j = 0; j < cols; j++) {
            transposed[j][i] = original[i][j];
        }
    }
}

// Function to print a matrix
void printMatrix(int matrix[][3], int rows, int cols) {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }
}

// Overloaded function to print the transposed matrix
void printTransposed(int matrix[][2], int rows, int cols) {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            printf("%d ", matrix[i][j]);
        }
        printf("\n");
    }
}

int main() {
    int original[2][3] = {
        {1, 2, 3},
        {4, 5, 6}
    };

    int transposed[3][2];

    printf("Original Matrix (2x3):\n");
    printMatrix(original, 2, 3);

    transposeMatrix(original, transposed, 2, 3);

    printf("\nTransposed Matrix (3x2):\n");
    printTransposed(transposed, 3, 2);

    return 0;
}

```

Problem 3: String Manipulation using Pointers

```

#include <stdio.h>

// Function to find the length of a string using pointers
int stringLength(const char *str) {

```

```

    const char *ptr = str;
    while (*ptr != '\0') {
        ptr++;
    }
    return ptr - str;
}

// Function to copy a string using pointers
void stringCopy(char *dest, const char *src) {
    while (*src != '\0') {
        *dest = *src;
        dest++;
        src++;
    }
    *dest = '\0'; // Add null terminator
}

// Function to concatenate two strings using pointers
void stringConcat(char *dest, const char *src) {
    // Move to the end of dest
    while (*dest != '\0') {
        dest++;
    }

    // Copy src to the end of dest
    while (*src != '\0') {
        *dest = *src;
        dest++;
        src++;
    }
    *dest = '\0'; // Add null terminator
}

int main() {
    char str1[50] = "Hello";
    char str2[] = "World";
    char result[50];

    printf("str1: %s\n", str1);
    printf("str2: %s\n", str2);

    printf("Length of str1: %d\n", strlen(str1));

    stringCopy(result, str1);
    printf("After copying str1 to result: %s\n", result);

    stringConcat(result, " ");
    stringConcat(result, str2);
    printf("After concatenating: %s\n", result);

    return 0;
}

```


Problem 4: Array of Pointers to Structs

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    int id;
    char name[50];
    float salary;
} Employee;

// Function to display employee details
void displayEmployee(const Employee *emp) {
    printf("ID: %d, Name: %s, Salary: $%.2f\n",
        emp->id, emp->name, emp->salary);
}

int main() {
    // Create an array of Employee structures
    Employee employees[3] = {
        {101, "John Smith", 45000.0f},
        {102, "Alice Johnson", 52000.0f},
        {103, "Bob Williams", 48000.0f}
    };

    // Create an array of pointers to employees
    Employee *empPtrs[3];
    for (int i = 0; i < 3; i++) {
        empPtrs[i] = &employees[i];
    }

    printf("Employee Details:\n");
    for (int i = 0; i < 3; i++) {
        displayEmployee(empPtrs[i]);
    }

    // Sort employees by salary (using the pointers)
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2 - i; j++) {
            if (empPtrs[j]->salary > empPtrs[j+1]->salary) {
                // Swap pointers
                Employee *temp = empPtrs[j];
                empPtrs[j] = empPtrs[j+1];
                empPtrs[j+1] = temp;
            }
        }
    }

    printf("\nEmployees Sorted by Salary:\n");
    for (int i = 0; i < 3; i++) {
        displayEmployee(empPtrs[i]);
    }
}
```

```
}

return 0;

}
```

Problem Solving using Array of Pointers and Functions

Problem 1: Menu-Driven Program using Function Pointers

```
#include <stdio.h>
#include <stdlib.h>

// Function to find the sum of array elements
void findSum(int arr[], int size) {
    int sum = 0;
    for (int i = 0; i < size; i++) {
        sum += arr[i];
    }
    printf("Sum of elements: %d\n", sum);
}

// Function to find the average of array elements
void findAverage(int arr[], int size) {
    if (size == 0) {
        printf("Cannot calculate average of empty array\n");
        return;
    }

    int sum = 0;
    for (int i = 0; i < size; i++) {
        sum += arr[i];
    }
    printf("Average of elements: %.2f\n", (float)sum / size);
}

// Function to find the maximum element
void findMax(int arr[], int size) {
    if (size == 0) {
        printf("Array is empty\n");
        return;
    }

    int max = arr[0];
    for (int i = 1; i < size; i++) {
        if (arr[i] > max) {
            max = arr[i];
        }
    }
    printf("Maximum element: %d\n", max);
}
```

```

// Function to find the minimum element
void findMin(int arr[], int size) {
    if (size == 0) {
        printf("Array is empty\n");
        return;
    }

    int min = arr[0];
    for (int i = 1; i < size; i++) {
        if (arr[i] < min) {
            min = arr[i];
        }
    }
    printf("Minimum element: %d\n", min);
}

int main() {
    int numbers[] = {12, 45, 67, 23, 9, 56, 89, 32};
    int size = sizeof(numbers) / sizeof(numbers[0]);

    // Array of function pointers
    void (*operations[])(int[], int) = {findSum, findAverage, findMax, findMin};
    char *op_names[] = {"Sum", "Average", "Maximum", "Minimum"};

    // Display menu
    printf("Array: ");
    for (int i = 0; i < size; i++) {
        printf("%d ", numbers[i]);
    }
    printf("\n\n");

    printf("Choose an operation:\n");
    printf("1. Calculate Sum\n");
    printf("2. Calculate Average\n");
    printf("3. Find Maximum\n");
    printf("4. Find Minimum\n");
    printf("0. Exit\n");

    int choice;
    do {
        printf("\nEnter your choice (0-4): ");
        scanf("%d", &choice);

        if (choice >= 1 && choice <= 4) {
            operations[choice-1](numbers, size);
        } else if (choice != 0) {
            printf("Invalid choice! Please try again.\n");
        }
    } while (choice != 0);

    printf("Program terminated.\n");
}

```

```
    return 0;
}
```

Problem 2: Text Processing with Function Pointers

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

// Function to convert text to uppercase
void toUpperCase(char *text) {
    while (*text) {
        *text = toupper(*text);
        text++;
    }
}

// Function to convert text to lowercase
void toLowerCase(char *text) {
    while (*text) {
        *text = tolower(*text);
        text++;
    }
}

// Function to capitalize first letter of each word
void capitalizeWords(char *text) {
    int capitalize = 1; // Flag to indicate if next character should be capitalized

    while (*text) {
        if (isspace(*text)) {
            capitalize = 1;
        } else if (capitalize) {
            *text = toupper(*text);
            capitalize = 0;
        } else {
            *text = tolower(*text);
        }
        text++;
    }
}

// Function to reverse the text
void reverseText(char *text) {
    char *start = text;
    char *end = text + strlen(text) - 1;

    while (start < end) {
        // Swap characters
        char temp = *start;
        *start = *end;
        *end = temp;
        start++;
        end--;
    }
}
```

```

        *end = temp;

        // Move pointers
        start++;
        end--;
    }
}

int main() {
    char text[100];
    char original[100];

    printf("Enter a text string: ");
    fgets(text, sizeof(text), stdin);

    // Remove newline character if present
    size_t len = strlen(text);
    if (len > 0 && text[len-1] == '\n') {
        text[len-1] = '\0';
    }

    // Save original text
    strcpy(original, text);

    // Array of function pointers
    void (*textOperations[])(char *) = {
        toUpperCase,
        toLowerCase,
        capitalizeWords,
        reverseText
    };

    char *op_names[] = {
        "Convert to UPPERCASE",
        "Convert to lowercase",
        "Capitalize First Letter Of Each Word",
        "Reverse the text"
    };

    // Display menu
    printf("\nText Operations Menu:\n");
    for (int i = 0; i < 4; i++) {
        printf("%d. %s\n", i+1, op_names[i]);
    }
    printf("0. Exit\n");

    int choice;
    do {
        printf("\nEnter your choice (0-4): ");
        scanf("%d", &choice);

        if (choice >= 1 && choice <= 4) {
            // Reset text to original

```

```

        strcpy(text, original);

        // Apply the selected operation
        textOperations[choice-1](text);

        // Display the result
        printf("Result: %s\n", text);
    } else if (choice != 0) {
        printf("Invalid choice! Please try again.\n");
    }
} while (choice != 0);

printf("Program terminated.\n");

return 0;
}

```

Storage Classes

Storage classes in C determine the scope, lifetime, and initialization of variables.

auto Storage Class

The `auto` storage class is the default for local variables. Variables are automatically created at the beginning of a block and destroyed at the end.

```

#include <stdio.h>

void demoAutoStorage() {
    auto int x = 10; // 'auto' is optional, as it's the default
    printf("Inside function: x = %d\n", x);
}

int main() {
    int y = 20; // 'auto' is implied
    {
        auto int z = 30;
        printf("Inside block: y = %d, z = %d\n", y, z);
    }
    // z is no longer accessible here

    demoAutoStorage();
    // x from demoAutoStorage() is not accessible here

    return 0;
}

```

register Storage Class

The `register` storage class suggests the compiler to store the variable in a CPU register for faster access. However, modern compilers optimize well, so this is often unnecessary.

```
#include <stdio.h>
#include <time.h>

int main() {
    // Example with register variable for loop counter
    clock_t start, end;
    double cpu_time_used;

    start = clock();
    register int i; // Suggest compiler to use a CPU register
    int sum = 0;
    for (i = 0; i < 1000000; i++) {
        sum += i;
    }
    end = clock();
    cpu_time_used = ((double) (end - start)) / CLOCKS_PER_SEC;

    printf("Sum: %d\n", sum);
    printf("Time taken: %f seconds\n", cpu_time_used);

    return 0;
}
```

static Storage Class

The `static` storage class has two uses:

1. For local variables, it preserves the value between function calls.
2. For global variables and functions, it limits the scope to the file.

```
#include <stdio.h>

// Function using static variable
void countCalls() {
    static int count = 0; // Initialized only once
    count++;
    printf("This function has been called %d time(s)\n", count);
}

// Static global variable
static int fileScope = 100; // Only accessible within this file

int main() {
    printf("Initial value of fileScope: %d\n", fileScope);

    // Demonstrate static local variable
```

```
    for (int i = 0; i < 5; i++) {  
        countCalls();  
    }  
  
    return 0;  
}
```

extern Storage Class

The `extern` storage class declares a variable that is defined elsewhere (typically in another file).

File: main.c

```
#include <stdio.h>  
  
// External variable declared but not defined here  
extern int sharedCounter;  
  
// External function declaration  
extern void incrementCounter();  
  
int main() {  
    printf("Initial counter value: %d\n", sharedCounter);  
  
    incrementCounter();  
    incrementCounter();  
  
    printf("Final counter value: %d\n", sharedCounter);  
  
    return 0;  
}
```

File: counter.c

```
// Variable definition  
int sharedCounter = 0;  
  
// Function definition  
void incrementCounter() {  
    sharedCounter++;  
}
```

Callback

A callback is a function that is passed as an argument to another function and is invoked at a specific time or when a specific event occurs.

Basic Callback Example

```
#include <stdio.h>

// Callback function type
typedef void (*CallbackFunc)(int);

// Functions that will be used as callbacks
void printSquare(int num) {
    printf("Square of %d is %d\n", num, num * num);
}

void printCube(int num) {
    printf("Cube of %d is %d\n", num, num * num * num);
}

// Function that takes a callback
void processNumbers(int arr[], int size, CallbackFunc callback) {
    for (int i = 0; i < size; i++) {
        callback(arr[i]);
    }
}

int main() {
    int numbers[] = {1, 2, 3, 4, 5};
    int size = sizeof(numbers) / sizeof(numbers[0]);

    printf("Processing with square callback:\n");
    processNumbers(numbers, size, printSquare);

    printf("\nProcessing with cube callback:\n");
    processNumbers(numbers, size, printCube);

    return 0;
}
```

Advanced Callback Example with Context

```
#include <stdio.h>

// Structure to hold callback context
typedef struct {
    void (*callback)(void *data, int value);
    void *data;
} CallbackInfo;

// Callback functions
void multiplyBy(void *factor, int value) {
    int multiplier = *((int *)factor);
    printf("%d * %d = %d\n", value, multiplier, value * multiplier);
}
```

```

}

void powerOf(void *exponent, int value) {
    int exp = *((int *)exponent);
    int result = 1;
    for (int i = 0; i < exp; i++) {
        result *= value;
    }
    printf("%d^%d = %d\n", value, exp, result);
}

// Function that uses the callback info
void processWithContext(int arr[], int size, CallbackInfo cbInfo) {
    for (int i = 0; i < size; i++) {
        cbInfo.callback(cbInfo.data, arr[i]);
    }
}

int main() {
    int numbers[] = {1, 2, 3, 4, 5};
    int size = sizeof(numbers) / sizeof(numbers[0]);

    // Set up callback with context for multiplication
    int factor = 10;
    CallbackInfo multiplyInfo = {multiplyBy, &factor};

    printf("Multiplying by %d:\n", factor);
    processWithContext(numbers, size, multiplyInfo);

    // Set up callback with context for exponentiation
    int exponent = 3;
    CallbackInfo powerInfo = {powerOf, &exponent};

    printf("\nRaising to power %d:\n", exponent);
    processWithContext(numbers, size, powerInfo);

    return 0;
}

```

Recursion

Recursion is a technique where a function calls itself to solve a problem.

Basic Recursion Example: Factorial

```

#include <stdio.h>

// Recursive function to calculate factorial
unsigned long factorial(int n) {
    // Base case

```

```
    if (n == 0 || n == 1) {
        return 1;
    }

    // Recursive case
    return n * factorial(n - 1);
}

int main() {
    for (int i = 0; i <= 10; i++) {
        printf("Factorial of %d = %lu\n", i, factorial(i));
    }

    return 0;
}
```

Fibonacci Sequence using Recursion

```
#include <stdio.h>

// Recursive function to calculate Fibonacci number
int fibonacci(int n) {
    // Base cases
    if
```

Recursion, Searching, Sorting, and Callbacks in C Programming

Table of Contents

1. [Introduction to Recursion](#)
2. [Problem Solving using Recursion](#)
3. [Searching Algorithms](#)
4. [Sorting Algorithms](#)
5. [Problem Solving using Callbacks](#)

Introduction to Recursion

Recursion is a programming technique where a function calls itself to solve a problem. The solution to a complex problem is expressed in terms of simpler instances of the same problem.

Key Components of Recursion:

1. **Base Case:** The condition that stops the recursion
2. **Recursive Case:** The part where the function calls itself

Basic Syntax of a Recursive Function in C:

```
return_type function_name(parameters) {  
    // Base case  
    if (base_case_condition) {  
        return base_case_value;  
    }  
    // Recursive case  
    else {  
        // Process data and make recursive call(s)  
        return function_name(modified_parameters);  
    }  
}
```

Example: Factorial Calculation

```
#include <stdio.h>  
  
// Recursive function to calculate factorial  
unsigned long factorial(unsigned int n) {  
    // Base case  
    if (n == 0 || n == 1) {  
        return 1;  
    }  
    // Recursive case  
    else {  
        return n * factorial(n - 1);  
    }  
}  
  
int main() {  
    unsigned int number = 5;  
    printf("Factorial of %u is %lu\n", number, factorial(number));  
    return 0;  
}
```

Output:

```
Factorial of 5 is 120
```

In this example:

- The base case is when `n` equals 0 or 1, where the factorial is 1.

- The recursive case calculates `n * factorial(n-1)` .

Memory Stack During Recursion

Understanding what happens in memory is crucial:

For `factorial(5)` :

1. Call `factorial(5)` → Waits for `factorial(4)`
2. Call `factorial(4)` → Waits for `factorial(3)`
3. Call `factorial(3)` → Waits for `factorial(2)`
4. Call `factorial(2)` → Waits for `factorial(1)`
5. Call `factorial(1)` → Returns 1 (base case)
6. `factorial(2)` completes: $2 * 1 = 2$
7. `factorial(3)` completes: $3 * 2 = 6$
8. `factorial(4)` completes: $4 * 6 = 24$
9. `factorial(5)` completes: $5 * 24 = 120$

Recursion vs. Iteration

Recursion often provides a more elegant and natural solution for certain problems, but it comes with a cost:

Aspect	Recursion	Iteration
Memory	Higher (stack space)	Lower
Speed	Generally slower	Generally faster
Clarity	Often clearer for certain problems	Can be more complex for the same problems
Stack Overflow	Risk exists	No risk

Problem Solving using Recursion

Let's explore more complex problems that can be elegantly solved using recursion.

Example 1: Fibonacci Sequence

```
#include <stdio.h>

// Recursive function to find the nth Fibonacci number
int fibonacci(int n) {
```

```

// Base cases
if (n <= 0) {
    return 0;
}
if (n == 1) {
    return 1;
}

// Recursive case
return fibonacci(n - 1) + fibonacci(n - 2);
}

int main() {
    int n = 10;
    printf("The %dth Fibonacci number is: %d\n", n, fibonacci(n));

    printf("Fibonacci sequence up to %d:\n", n);
    for (int i = 0; i <= n; i++) {
        printf("%d ", fibonacci(i));
    }
    printf("\n");

    return 0;
}

```

Output:

```

The 10th Fibonacci number is: 55
Fibonacci sequence up to 10:
0 1 1 2 3 5 8 13 21 34 55

```

Note: Simple recursive Fibonacci is inefficient due to redundant calculations. For production code, dynamic programming or memoization should be used.

Example 2: Tower of Hanoi

The Tower of Hanoi is a classic recursion problem:

```

#include <stdio.h>

// Recursive function to solve Tower of Hanoi
void towerOfHanoi(int n, char source, char auxiliary, char destination) {
    // Base case
    if (n == 1) {
        printf("Move disk 1 from rod %c to rod %c\n", source, destination);
        return;
    }

    // Recursive cases

```

```

// Move n-1 disks from source to auxiliary using destination as auxiliary
towerOfHanoi(n - 1, source, destination, auxiliary);

// Move the nth disk from source to destination
printf("Move disk %d from rod %c to rod %c\n", n, source, destination);

// Move n-1 disks from auxiliary to destination using source as auxiliary
towerOfHanoi(n - 1, auxiliary, source, destination);
}

int main() {
    int n = 3; // Number of disks
    towerOfHanoi(n, 'A', 'B', 'C'); // A=source, B=auxiliary, C=destination
    return 0;
}

```

Output:

```

Move disk 1 from rod A to rod C
Move disk 2 from rod A to rod B
Move disk 1 from rod C to rod B
Move disk 3 from rod A to rod C
Move disk 1 from rod B to rod A
Move disk 2 from rod B to rod C
Move disk 1 from rod A to rod C

```

Example 3: Binary Tree Traversal

Recursion is natural for tree traversals:

```

#include <stdio.h>
#include <stdlib.h>

// Binary tree node structure
struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};

// Function to create a new node
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->left = NULL;
    newNode->right = NULL;
    return newNode;
}

```

```

// Recursive inorder traversal
void inorderTraversal(struct Node* root) {
    // Base case
    if (root == NULL) {
        return;
    }

    // Recursive cases
    inorderTraversal(root->left); // Traverse left subtree
    printf("%d ", root->data);    // Visit node
    inorderTraversal(root->right); // Traverse right subtree
}

// Recursive preorder traversal
void preorderTraversal(struct Node* root) {
    if (root == NULL) {
        return;
    }

    printf("%d ", root->data); // Visit node
    preorderTraversal(root->left); // Traverse left subtree
    preorderTraversal(root->right); // Traverse right subtree
}

// Recursive postorder traversal
void postorderTraversal(struct Node* root) {
    if (root == NULL) {
        return;
    }

    postorderTraversal(root->left); // Traverse left subtree
    postorderTraversal(root->right); // Traverse right subtree
    printf("%d ", root->data);    // Visit node
}

int main() {
    // Create a sample binary tree
    struct Node* root = createNode(1);
    root->left = createNode(2);
    root->right = createNode(3);
    root->left->left = createNode(4);
    root->left->right = createNode(5);

    printf("Inorder traversal: ");
    inorderTraversal(root);
    printf("\n");

    printf("Preorder traversal: ");
    preorderTraversal(root);
    printf("\n");

    printf("Postorder traversal: ");
    postorderTraversal(root);
}

```



```

printf("\n");

// Free memory (not shown in detail)
// Would need a recursive free function

return 0;
}

```

Output:

```

Inorder traversal: 4 2 5 1 3
Preorder traversal: 1 2 4 5 3
Postorder traversal: 4 5 2 3 1

```

Searching Algorithms

Searching is the process of finding a specific element within a collection of data. Let's implement the two most common searching algorithms.

Linear Search

Linear search sequentially checks each element until the target is found or the end is reached.

```

#include <stdio.h>

// Recursive linear search
int recursiveLinearSearch(int arr[], int size, int target, int currentIndex) {
    // Base cases
    if (currentIndex >= size) { // Element not found
        return -1;
    }
    if (arr[currentIndex] == target) { // Element found
        return currentIndex;
    }

    // Recursive case
    return recursiveLinearSearch(arr, size, target, currentIndex + 1);
}

// Iterative linear search
int iterativeLinearSearch(int arr[], int size, int target) {
    for (int i = 0; i < size; i++) {
        if (arr[i] == target) {
            return i; // Return index of the found element
        }
    }
    return -1; // Element not found
}

```

```

int main() {
    int arr[] = {12, 34, 54, 2, 3};
    int size = sizeof(arr) / sizeof(arr[0]);
    int target = 3;

    // Using recursive linear search
    int recResult = recursiveLinearSearch(arr, size, target, 0);
    if (recResult != -1) {
        printf("Recursive Linear Search: Element found at index %d\n", recResult);
    } else {
        printf("Recursive Linear Search: Element not found\n");
    }

    // Using iterative linear search
    int iterResult = iterativeLinearSearch(arr, size, target);
    if (iterResult != -1) {
        printf("Iterative Linear Search: Element found at index %d\n", iterResult);
    } else {
        printf("Iterative Linear Search: Element not found\n");
    }

    return 0;
}

```

Output:

```

Recursive Linear Search: Element found at index 4
Iterative Linear Search: Element found at index 4

```

Binary Search

Binary search is much faster than linear search for sorted arrays, with $O(\log n)$ time complexity.

```

#include <stdio.h>

// Recursive binary search
int recursiveBinarySearch(int arr[], int low, int high, int target) {
    // Base case
    if (low > high) {
        return -1; // Element not found
    }

    // Calculate middle index
    int mid = low + (high - low) / 2;

    // Found the element
    if (arr[mid] == target) {
        return mid;
    }
}

```

```

}

// Recursive cases
// Search in the left half
if (arr[mid] > target) {
    return recursiveBinarySearch(arr, low, mid - 1, target);
}

// Search in the right half
return recursiveBinarySearch(arr, mid + 1, high, target);
}

// Iterative binary search
int iterativeBinarySearch(int arr[], int size, int target) {
    int low = 0;
    int high = size - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        // Check if target is present at mid
        if (arr[mid] == target) {
            return mid;
        }

        // If target is greater, ignore left half
        if (arr[mid] < target) {
            low = mid + 1;
        }
        // If target is smaller, ignore right half
        else {
            high = mid - 1;
        }
    }

    return -1; // Element not found
}

int main() {
    int arr[] = {2, 3, 12, 34, 54, 65, 78}; // Must be sorted for binary search
    int size = sizeof(arr) / sizeof(arr[0]);
    int target = 34;

    // Using recursive binary search
    int recResult = recursiveBinarySearch(arr, 0, size - 1, target);
    if (recResult != -1) {
        printf("Recursive Binary Search: Element found at index %d\n", recResult);
    } else {
        printf("Recursive Binary Search: Element not found\n");
    }

    // Using iterative binary search
    int iterResult = iterativeBinarySearch(arr, size, target);

```

```

    if (iterResult != -1) {
        printf("Iterative Binary Search: Element found at index %d\n", iterResult);
    } else {
        printf("Iterative Binary Search: Element not found\n");
    }

    return 0;
}

```

Output:

```

Recursive Binary Search: Element found at index 3
Iterative Binary Search: Element found at index 3

```

Sorting Algorithms

Sorting arranges elements in a specific order. Let's implement several common sorting algorithms using both recursive and iterative approaches.

Quick Sort (Recursive)

Quick sort is an efficient, divide-and-conquer sorting algorithm with an average time complexity of $O(n \log n)$.

```

#include <stdio.h>

// Function to swap two elements
void swap(int* a, int* b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Function to partition the array
int partition(int arr[], int low, int high) {
    int pivot = arr[high]; // Select the rightmost element as pivot
    int i = (low - 1);      // Index of smaller element

    for (int j = low; j <= high - 1; j++) {
        // If current element is smaller than or equal to pivot
        if (arr[j] <= pivot) {
            i++; // Increment index of smaller element
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return (i + 1);
}

```

```

// Recursive QuickSort function
void quickSort(int arr[], int low, int high) {
    if (low < high) {
        // pi is the partitioning index
        int pi = partition(arr, low, high);

        // Sort elements before and after partition
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

// Function to print an array
void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main() {
    int arr[] = {10, 7, 8, 9, 1, 5};
    int size = sizeof(arr) / sizeof(arr[0]);

    printf("Original array: ");
    printArray(arr, size);

    quickSort(arr, 0, size - 1);

    printf("Sorted array: ");
    printArray(arr, size);

    return 0;
}

```

Output:

```

Original array: 10 7 8 9 1 5
Sorted array: 1 5 7 8 9 10

```

Merge Sort (Recursive)

Merge sort is a stable, divide-and-conquer algorithm with a time complexity of $O(n \log n)$.

```

#include <stdio.h>
#include <stdlib.h>

// Merge two subarrays of arr[]

```

```

void merge(int arr[], int left, int mid, int right) {
    int i, j, k;
    int n1 = mid - left + 1;
    int n2 = right - mid;

    // Create temporary arrays
    int *L = (int*)malloc(n1 * sizeof(int));
    int *R = (int*)malloc(n2 * sizeof(int));

    // Copy data to temporary arrays
    for (i = 0; i < n1; i++) {
        L[i] = arr[left + i];
    }
    for (j = 0; j < n2; j++) {
        R[j] = arr[mid + 1 + j];
    }

    // Merge temporary arrays back into arr[left..right]
    i = 0;        // Initial index of first subarray
    j = 0;        // Initial index of second subarray
    k = left;     // Initial index of merged subarray

    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

    // Copy remaining elements of L[] if any
    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }

    // Copy remaining elements of R[] if any
    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }

    // Free the allocated memory
    free(L);
    free(R);
}

// Recursive merge sort function

```

```

void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        // Find the middle point
        int mid = left + (right - left) / 2;

        // Sort first and second halves
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        // Merge the sorted halves
        merge(arr, left, mid, right);
    }
}

// Function to print an array
void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main() {
    int arr[] = {12, 11, 13, 5, 6, 7};
    int size = sizeof(arr) / sizeof(arr[0]);

    printf("Original array: ");
    printArray(arr, size);

    mergeSort(arr, 0, size - 1);

    printf("Sorted array: ");
    printArray(arr, size);

    return 0;
}

```

Output:

```

Original array: 12 11 13 5 6 7
Sorted array: 5 6 7 11 12 13

```

Bubble Sort (Recursive)

Bubble sort is a simple comparison-based algorithm. Here's a recursive implementation:

```

#include <stdio.h>

// A recursive function for bubble sort

```

```

void recursiveBubbleSort(int arr[], int n) {
    // Base case
    if (n == 1) {
        return;
    }

    // One pass of bubble sort. After this pass, the largest element is moved to the end
    int swapped = 0;
    for (int i = 0; i < n - 1; i++) {
        if (arr[i] > arr[i + 1]) {
            // Swap arr[i] and arr[i+1]
            int temp = arr[i];
            arr[i] = arr[i + 1];
            arr[i + 1] = temp;
            swapped = 1;
        }
    }

    // If no swapping occurred in this pass, array is sorted
    if (swapped == 0) {
        return;
    }

    // Largest element is fixed, recur for remaining array
    recursiveBubbleSort(arr, n - 1);
}

// Function to print an array
void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main() {
    int arr[] = {64, 34, 25, 12, 22, 11, 90};
    int size = sizeof(arr) / sizeof(arr[0]);

    printf("Original array: ");
    printArray(arr, size);

    recursiveBubbleSort(arr, size);

    printf("Sorted array: ");
    printArray(arr, size);

    return 0;
}

```

Output:

Original array: 64 34 25 12 22 11 90

Sorted array: 11 12 22 25 34 64 90

Problem Solving using Callbacks

Callbacks are function pointers in C that allow passing functions as arguments to other functions. They're powerful for creating flexible, reusable code.

Understanding Function Pointers

First, let's understand function pointers:

```
#include <stdio.h>

// A simple function
int add(int a, int b) {
    return a + b;
}

// Another simple function
int multiply(int a, int b) {
    return a * b;
}

int main() {
    // Declare a function pointer
    int (*operation)(int, int);

    // Assign function address to the pointer
    operation = add;
    printf("Result of addition: %d\n", operation(5, 3));

    // Change the function being pointed to
    operation = multiply;
    printf("Result of multiplication: %d\n", operation(5, 3));

    return 0;
}
```

Output:

```
Result of addition: 8
Result of multiplication: 15
```

Callbacks for Searching

Let's create a generic search function using callbacks:

```
#include <stdio.h>

// A search function that uses a callback to determine if an element matches
int findElement(int arr[], int size, int (*matchFunc)(int, int), int key) {
    for (int i = 0; i < size; i++) {
        if (matchFunc(arr[i], key)) {
            return i; // Return index if element is found
        }
    }
    return -1; // Return -1 if element is not found
}

// Callback for exact match
int exactMatch(int element, int key) {
    return element == key;
}

// Callback for finding elements greater than key
int greaterThan(int element, int key) {
    return element > key;
}

// Callback for finding even elements
int isEven(int element, int key) {
    // Key is ignored in this callback
    return element % 2 == 0;
}

int main() {
    int arr[] = {5, 12, 7, 20, 15, 8};
    int size = sizeof(arr) / sizeof(arr[0]);

    // Find exact match for 7
    int index = findElement(arr, size, exactMatch, 7);
    if (index != -1) {
        printf("Element 7 found at index: %d\n", index);
    } else {
        printf("Element 7 not found\n");
    }

    // Find first element greater than 10
    index = findElement(arr, size, greaterThan, 10);
    if (index != -1) {
        printf("First element greater than 10 found at index %d with value %d\n", index, arr[index]);
    } else {
        printf("No element greater than 10 found\n");
    }

    // Find first even element
    index = findElement(arr, size, isEven, 0); // Key is ignored in isEven
}
```

```

    if (index != -1) {
        printf("First even element found at index %d with value %d\n", index, arr[index]);
    } else {
        printf("No even element found\n");
    }

    return 0;
}

```

Output:

```

Element 7 found at index: 2
First element greater than 10 found at index 1 with value 12
First even element found at index 1 with value 12

```

Callbacks for Sorting

We can use callbacks to create a generic sorting function:

```

#include <stdio.h>

// Function to swap two integers
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// A generic bubble sort function that uses a callback to determine sort order
void bubbleSort(int arr[], int size, int (*compare)(int, int)) {
    for (int i = 0; i < size - 1; i++) {
        for (int j = 0; j < size - i - 1; j++) {
            // If compare function returns true, swap the elements
            if (compare(arr[j], arr[j + 1])) {
                swap(&arr[j], &arr[j + 1]);
            }
        }
    }
}

// Callback for ascending order
int ascending(int a, int b) {
    return a > b;
}

// Callback for descending order
int descending(int a, int b) {
    return a < b;
}

```

```

// Callback for even-first ordering (even numbers before odd numbers)
int evenFirst(int a, int b) {
    // If a is odd and b is even, swap them
    if (a % 2 != 0 && b % 2 == 0) {
        return 1;
    }
    // If both are even or both are odd, maintain original order
    else if ((a % 2 == 0 && b % 2 == 0) || (a % 2 != 0 && b % 2 != 0)) {
        return a > b; // Sort in ascending order within the same parity
    }
    // If a is even and b is odd, keep them as is
    return 0;
}

// Function to print an array
void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main() {
    int arr[] = {64, 34, 25, 12, 22, 11, 90};
    int size = sizeof(arr) / sizeof(arr[0]);
    int arrCopy1[size], arrCopy2[size];

    // Copy original array to preserve it for multiple sorts
    for (int i = 0; i < size; i++) {
        arrCopy1[i] = arr[i];
        arrCopy2[i] = arr[i];
    }

    printf("Original array: ");
    printArray(arr, size);

    // Sort in ascending order
    bubbleSort(arr, size, ascending);
    printf("Sorted in ascending order: ");
    printArray(arr, size);

    // Sort in descending order
    bubbleSort(arrCopy1, size, descending);
    printf("Sorted in descending order: ");
    printArray(arrCopy1, size);

    // Sort by even-first rule
    bubbleSort(arrCopy2, size, evenFirst);
    printf("Sorted with even numbers first: ");
    printArray(arrCopy2, size);
}

```

```
    return 0;
}
```

Output:

```
Original array: 64 34 25 12 22 11 90
Sorted in ascending order: 11 12 22 25 34 64 90
Sorted in descending order: 90 64 34 25 22 12 11
Sorted with even numbers first: 12 22 34 64 90 11 25
```

Callbacks for Counting and Filtering

Callbacks can be used for various data processing tasks:

```
#include <stdio.h>
#include <stdlib.h>

// Count elements in array that match a certain condition
int countElements(int arr[], int size, int (*condition)(int)) {
    int count = 0;
    for (int i = 0; i < size; i++) {
        if (condition(arr[i])) {
            count++;
        }
    }
    return count;
}

// Filter elements into a new array based on a condition
int* filterElements(int arr[], int size, int (*condition)(int), int* resultSize) {
    // First, count how many elements will pass the filter
    *resultSize = countElements(arr, size, condition);

    // Allocate memory for the result array
    int* result = (int*)malloc(*resultSize * sizeof(int));

    // Copy elements that match the condition
    int j = 0;
    for (int i = 0; i < size; i++) {
        if (condition(arr[i])) {
            result[j++] = arr[i];
        }
    }

    return result;
}

// Condition functions
int isEven(int num) {
```

```

    return num % 2 == 0;
}

int isOdd(int num) {
    return num % 2 != 0;
}

int isPositive(int num) {
    return num > 0;
}

int isNegative(int num) {
    return num < 0;
}

int isPrime(int num) {
    if (num <= 1) return 0;
    if (num <= 3) return 1;
    if (num % 2 == 0 || num % 3 == 0) return 0;

    for (int i = 5; i * i <= num; i += 6) {
        if (num % i == 0 || num % (i + 2) == 0) {
            return 0;
        }
    }
    return 1;
}

// Function to print an array
void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main() {
    int arr[] = {-3, 2, 0, 7, -5, 11, 4, 6, -8, 9};
    int size = sizeof(arr) / sizeof(arr[0]);

    printf("Original array: ");
    printArray(arr, size);

    // Count even numbers
    printf("Number of even elements: %d\n", countElements(arr, size, isEven));

    // Count positive numbers
    printf("Number of positive elements: %d\n", countElements(arr, size, isPositive));

    // Count prime numbers
    printf("Number of prime elements: %d\n", countElements(arr, size, isPrime));

    // Filter even numbers

```

```

int evenSize;
int* evenNumbers = filterElements(arr, size, isEven, &evenSize);
printf("Even elements: ");
printArray(evenNumbers, evenSize);

// Filter prime numbers
int primeSize;
int* primeNumbers = filterElements(arr, size, isPrime, &primeSize);
printf("Prime elements: ");
printArray(primeNumbers, primeSize);

// Free allocated memory
free(evenNumbers);
free(primeNumbers);

return 0;
}

```

Output:

```

Original array: -3 2 0 7 -5 11 4 6 -8 9
Number of even elements: 5
Number of positive elements: 6
Number of prime elements: 4
Even elements: 2 0 4 6 -8
Prime elements: 2 7 11

```

Advanced Example: Map-Reduce with Callbacks

Let's implement a simple Map-Reduce pattern using callbacks:

```

#include

```

Advanced Example: Map-Reduce with Callbacks

Let's implement a simple Map-Reduce pattern using callbacks:

```

#include <stdio.h>
#include <stdlib.h>

// Map function: applies a transformation to each element in an array
int* mapArray(int arr[], int size, int (*transform)(int)) {
    int* result = (int*)malloc(size * sizeof(int));
    for (int i = 0; i < size; i++) {
        result[i] = transform(arr[i]);
    }
    return result;
}

```

```

}

// Reduce function: reduces an array to a single value using an accumulator
int reduceArray(int arr[], int size, int (*combine)(int, int), int initialValue) {
    int accumulator = initialValue;
    for (int i = 0; i < size; i++) {
        accumulator = combine(accumulator, arr[i]);
    }
    return accumulator;
}

// Transform functions for map
int square(int x) {
    return x * x;
}

int double_value(int x) {
    return x * 2;
}

int absolute(int x) {
    return x < 0 ? -x : x;
}

// Combine functions for reduce
int sum(int acc, int value) {
    return acc + value;
}

int product(int acc, int value) {
    return acc * value;
}

int max(int acc, int value) {
    return acc > value ? acc : value;
}

// Function to print an array
void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main() {
    int arr[] = {-3, 2, 0, 7, -5, 11, 4};
    int size = sizeof(arr) / sizeof(arr[0]);

    printf("Original array: ");
    printArray(arr, size);

    // Map examples

```



```

int* squares = mapArray(arr, size, square);
printf("Squares: ");
printArray(squares, size);

int* doubles = mapArray(arr, size, double_value);
printf("Doubled values: ");
printArray(doubles, size);

int* absolutes = mapArray(arr, size, absolute);
printf("Absolute values: ");
printArray(absolutes, size);

// Reduce examples
printf("Sum of elements: %d\n", reduceArray(arr, size, sum, 0));
printf("Product of elements: %d\n", reduceArray(arr, size, product, 1));
printf("Maximum element: %d\n", reduceArray(arr, size, max, arr[0]));

// Combine map and reduce
printf("Sum of squares: %d\n", reduceArray(squares, size, sum, 0));
printf("Sum of absolute values: %d\n", reduceArray(absolutes, size, sum, 0));

// Free allocated memory
free(squares);
free(doubles);
free(absolutes);

return 0;
}

```

Output:

```

Original array: -3 2 0 7 -5 11 4
Squares: 9 4 0 49 25 121 16
Doubled values: -6 4 0 14 -10 22 8
Absolute values: 3 2 0 7 5 11 4
Sum of elements: 16
Product of elements: 0
Maximum element: 11
Sum of squares: 224
Sum of absolute values: 32

```

Custom Data Processing Example

This example shows how to use callbacks for creating custom data processing pipelines:

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

```

```

// Define a structure for a person
typedef struct {
    char name[50];
    int age;
    float height; // in meters
    float weight; // in kg
} Person;

// Calculate BMI (Body Mass Index)
float calculateBMI(Person person) {
    return person.weight / (person.height * person.height);
}

// Filter callback type for people
typedef int (*PersonFilter)(Person);

// Map callback type for people
typedef void (*PersonTransform)(Person*);

// Print callback
typedef void (*PersonPrinter)(Person);

// Count people that match a condition
int countPeople(Person people[], int size, PersonFilter filter) {
    int count = 0;
    for (int i = 0; i < size; i++) {
        if (filter(people[i])) {
            count++;
        }
    }
    return count;
}

// Filter people into a new array
Person* filterPeople(Person people[], int size, PersonFilter filter, int* resultSize) {
    *resultSize = countPeople(people, size, filter);
    Person* result = (Person*)malloc(*resultSize * sizeof(Person));

    int j = 0;
    for (int i = 0; i < size; i++) {
        if (filter(people[i])) {
            result[j++] = people[i];
        }
    }

    return result;
}

// Transform each person in the array
void transformPeople(Person people[], int size, PersonTransform transform) {
    for (int i = 0; i < size; i++) {
        transform(&people[i]);
    }
}

```

```

}

// Print people using a custom printer
void printPeople(Person people[], int size, PersonPrinter printer) {
    for (int i = 0; i < size; i++) {
        printer(people[i]);
    }
}

// Filter functions
int isAdult(Person p) {
    return p.age >= 18;
}

int isOverweight(Person p) {
    // BMI over a threshold
    float bmi = calculateBMI(p);
    return bmi >= 25.0;
}

int isTall(Person p) {
    return p.height >= 1.8; // 1.8m or taller
}

// Transform functions
void adjustWeight(Person* p) {
    // Simulate weight change
    p->weight = p->weight * 0.95; // 5% loss
}

void growOlder(Person* p) {
    p->age++;
}

// Print functions
void printBasic(Person p) {
    printf("Name: %s, Age: %d\n", p.name, p.age);
}

void printDetailed(Person p) {
    printf("Name: %s, Age: %d, Height: %.2fm, Weight: %.1fkg, BMI: %.1f\n",
        p.name, p.age, p.height, p.weight, calculateBMI(p));
}

int main() {
    // Sample data
    Person people[] = {
        {"Alice", 25, 1.65, 55.0},
        {"Bob", 17, 1.82, 70.0},
        {"Charlie", 42, 1.78, 95.0},
        {"David", 30, 1.90, 85.0},
        {"Eve", 22, 1.60, 50.0}
    };
}

```

```

    int size = sizeof(people) / sizeof(people[0]);

    printf("All people (basic info):\n");
    printPeople(people, size, printBasic);

    printf("\nAll people (detailed info):\n");
    printPeople(people, size, printDetailed);

    // Filter examples
    int adultCount;
    Person* adults = filterPeople(people, size, isAdult, &adultCount);
    printf("\nAdults only:\n");
    printPeople(adults, adultCount, printDetailed);

    int tallCount;
    Person* tallPeople = filterPeople(people, size, isTall, &tallCount);
    printf("\nTall people only:\n");
    printPeople(tallPeople, tallCount, printDetailed);

    // Transform example
    printf("\nAfter weight adjustment:\n");
    transformPeople(people, size, adjustWeight);
    printPeople(people, size, printDetailed);

    // Free allocated memory
    free(adults);
    free(tallPeople);

    return 0;
}

```

Output:

All people (basic info):

Name: Alice, Age: 25
 Name: Bob, Age: 17
 Name: Charlie, Age: 42
 Name: David, Age: 30
 Name: Eve, Age: 22

All people (detailed info):

Name: Alice, Age: 25, Height: 1.65m, Weight: 55.0kg, BMI: 20.2
 Name: Bob, Age: 17, Height: 1.82m, Weight: 70.0kg, BMI: 21.1
 Name: Charlie, Age: 42, Height: 1.78m, Weight: 95.0kg, BMI: 30.0
 Name: David, Age: 30, Height: 1.90m, Weight: 85.0kg, BMI: 23.5
 Name: Eve, Age: 22, Height: 1.60m, Weight: 50.0kg, BMI: 19.5

Adults only:

Name: Alice, Age: 25, Height: 1.65m, Weight: 55.0kg, BMI: 20.2
 Name: Charlie, Age: 42, Height: 1.78m, Weight: 95.0kg, BMI: 30.0
 Name: David, Age: 30, Height: 1.90m, Weight: 85.0kg, BMI: 23.5

Name: Eve, Age: 22, Height: 1.60m, Weight: 50.0kg, BMI: 19.5

Tall people only:

Name: Bob, Age: 17, Height: 1.82m, Weight: 70.0kg, BMI: 21.1

Name: David, Age: 30, Height: 1.90m, Weight: 85.0kg, BMI: 23.5

After weight adjustment:

Name: Alice, Age: 25, Height: 1.65m, Weight: 52.2kg, BMI: 19.2

Name: Bob, Age: 17, Height: 1.82m, Weight: 66.5kg, BMI: 20.1

Name: Charlie, Age: 42, Height: 1.78m, Weight: 90.2kg, BMI: 28.5

Name: David, Age: 30, Height: 1.90m, Weight: 80.8kg, BMI: 22.4

Name: Eve, Age: 22, Height: 1.60m, Weight: 47.5kg, BMI: 18.6

Common Pitfalls and Best Practices

Recursion Pitfalls

1. **Stack Overflow:** Recursive functions can cause stack overflow if there are too many recursive calls or if the base case is not properly defined.

```
// Dangerous recursion - may cause stack overflow
int dangerous_recursion(int n) {
    // Missing or incorrect base case
    return n + dangerous_recursion(n - 1);
}
```

2. **Inefficient Repeated Calculations:** As seen in the naive Fibonacci implementation, recursion can lead to many redundant calculations.
3. **Memory Usage:** Each recursive call adds a new layer to the call stack, consuming memory.

Best Practices for Recursion

1. **Always Define a Proper Base Case:** Ensure there's always a way to terminate the recursion.

```
int safe_recursion(int n) {
    // Proper base case
    if (n <= 0) {
        return 0;
    }
    return n + safe_recursion(n - 1);
}
```

2. **Consider Tail Recursion:** Tail recursion can be optimized by many compilers.

```
// Tail-recursive factorial function
unsigned long factorial_tail(unsigned int n, unsigned long accumulator) {
    if (n == 0 || n == 1) {
        return accumulator;
    }
}
```

```

    }
    return factorial_tail(n - 1, n * accumulator);
}

// Wrapper function
unsigned long factorial(unsigned int n) {
    return factorial_tail(n, 1);
}

```

3. **Use Memoization for Expensive Calculations:** Store results of subproblems to avoid redundant calculations.

```

#include <stdio.h>
#include <stdlib.h>

// Fibonacci with memoization
int fibonacci_memo(int n, int* memo) {
    // Base cases
    if (n <= 0) return 0;
    if (n == 1) return 1;

    // Check if already computed
    if (memo[n] != -1) {
        return memo[n];
    }

    // Compute and store
    memo[n] = fibonacci_memo(n - 1, memo) + fibonacci_memo(n - 2, memo);
    return memo[n];
}

int fibonacci(int n) {
    // Initialize memoization array
    int* memo = (int*)malloc((n + 1) * sizeof(int));
    for (int i = 0; i <= n; i++) {
        memo[i] = -1;
    }

    int result = fibonacci_memo(n, memo);
    free(memo);
    return result;
}

```

4. **Consider Iterative Alternatives:** For some problems, an iterative solution might be more efficient.

Callback Best Practices

1. **Type Safety:** Use typedefs for callback function pointers to improve clarity and safety.

```

typedef int (*ComparisonFunction)(int, int);

```

```
void bubbleSort(int arr[], int size, ComparisonFunction compare) {
    // Implementation
}
```

2. **Error Handling:** Check if callback functions are valid before using them.

```
void safeFunction(int arr[], int size, int (*callback)(int)) {
    if (callback == NULL) {
        printf("Error: Null callback function\n");
        return;
    }
    // Use callback
}
```

3. **Context Data:** Pass additional context data to callbacks when needed.

```
typedef struct {
    // Context data
    int threshold;
    int multiplier;
} Context;

int transformWithContext(int value, Context* ctx) {
    if (value > ctx->threshold) {
        return value * ctx->multiplier;
    }
    return value;
}

void processWithContext(int arr[], int size, int (*transform)(int, Context*), Context* ctx) {
    for (int i = 0; i < size; i++) {
        arr[i] = transform(arr[i], ctx);
    }
}
```

Conclusion

This document has covered fundamental concepts in C programming related to recursion, searching algorithms, sorting algorithms, and the use of callbacks for problem-solving. These techniques form the building blocks of many advanced algorithms and data structures.

Recursion offers an elegant way to solve problems by breaking them down into smaller instances of the same problem. While it can lead to cleaner code, developers must be mindful of its memory and performance implications.

Searching and sorting algorithms are essential for efficient data manipulation, and understanding their recursive implementations helps in grasping their underlying principles.

Callbacks provide a powerful mechanism for writing flexible and reusable code in C. They allow passing behavior as arguments, enabling generic functions that can be customized for specific needs.

By combining these techniques, C programmers can create efficient, maintainable, and flexible code to solve a wide variety of problems.